SHERLOCK

Security Review For Venus Protocol



Collaborative Audit Prepared For: **Venus Protocol**
Lead Security Expert(s): **KupiaSec**
TessKimy
Date Audited: **September 22 - September 28, 2026**

Introduction

Venus Spoke Pools are a new type of lending pool for Venus Protocol. They are built on the Venus isolated-pools contracts, the lending engine that already runs the Venus Core Pool on every supported network except BNB Chain.

In this engine, each pool stands on its own. A pool has its own `Comptroller`, which holds the pool's risk rules, and its own markets (`VTokens`), and it is listed in a `PoolRegistry`. An account's collateral in one pool cannot back its debt in another.

A spoke pool is such a pool, with its own comptroller, `SpokeComptroller`, and its own `PoolRegistry`. Its markets use the same `VToken` contract as other pools. Spoke pools are designed to be funded by the Venus Liquidity Hub, a per-asset ERC-4626 vault that allocates lender deposits across yield sources. The Hub supplies the assets that borrowers take, and anyone with eligible collateral can borrow.

`SpokeComptroller` makes the following changes to the isolated-pools `Comptroller`:

- **Supply allowlist per market.** Governance can restrict who supplies to a market. Only supplying is restricted. Redeeming is never blocked, so an account removed from the list can still withdraw.
- **Liquidation allowlist per pool.** Governance can restrict who seizes collateral. It covers regular liquidations, `liquidateAccount` and `healAccount`. Both allowlists are off by default.
- **Liquidation incentive per market.** Each collateral market can have its own incentive, and markets without one use the pool-wide value. A market's incentive must be at least `1e18` plus its protocol seize share, so a liquidator always receives collateral worth at least the debt it repaid. The pool-wide value must be at least `1.05e18`, the floor for a market at the default 5% seize share. `healAccount` and `liquidateAccount` use each market's own incentive.
- **Bounded prices for borrowing power.** Collateral-factor checks (borrow, redeem, transfer and exit market) read prices from a `DeviationBoundedOracle`. While protection is active for an asset, collateral is valued at the low end of the asset's recent price range and debt at the high end, so a price spike cannot raise borrowing power. Liquidations still use the spot price.
- **Entering a market for another account.** A router with an access-control role can enable a market as collateral for a supplier, so supplying and enabling collateral fit in one transaction.
- **Cleanup.** The Prime integration is removed, and revert strings are replaced with custom errors.

`ProtocolShareReserve`, which collects pool income, makes one change:

- **More than one pool registry.** It used to accept income only from the Core Pool and from markets in a single `PoolRegistry`. It now also checks an owner-managed list of additional registries, so it accepts income from markets in the spoke `PoolRegistry`.

The Liquidity Hub and the `DeviationBoundedOracle` are outside the scope of this audit. The audited code is at isolated-pools@7b02a608 and protocol-reserve@a7b8f12d.

Scope

Repository: VenusProtocol/isolated-pools

Audited Commit: 7b02a6082d213c01dc51614baba47f520eedf193

Final Commit: 01714a3b8a47383d87bea329c090b0b8affa3bd6

Files:

- contracts/Spoke/SpokeComptrollerInterface.sol
- contracts/Spoke/SpokeComptroller.sol
- contracts/Spoke/SpokeComptrollerStorage.sol

Repository: VenusProtocol/protocol-reserve

Audited Commit: a7b8f12de27657378d4173432852c70b000339e2

Final Commit: 4e589e25b99dcdf65bd72e63be06271d828da560

Files:

- contracts/ProtocolReserve/ProtocolShareReserve.sol

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	1	29

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue M-1: ProtocolShareReserve cannot release spoke pool income because RiskFundConverter checks only one pool registry [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit2/2026-09-venus-protocol-sep-11th-2026/issues/29

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

ProtocolShareReserve now accepts more than one pool registry, so spoke vTokens can send their income to it. But when that income is released, the PSR calls `updateAssetsState` on every destination, and RiskFundConverter still checks only its own single `poolRegistry`. That registry does not know the spoke comptroller, so `releaseFunds` reverts and the spoke income stays stuck in the PSR.

Vulnerability Details

Spoke markets are listed in `SpokePoolRegistry`. The PSR finds them through `additionalPoolRegistries`, so `updateAssetsState` from a spoke vToken works and the income is saved under `assetsReserves[spokeComptroller][asset]`.

To send this income out, anyone calls `releaseFunds(spokeComptroller, assets)`. For each destination, `_releaseFund` transfers the tokens and then calls `IIncomeDestination(destination).updateAssetsState(comptroller, asset)`.

RiskFundConverter checks the pool with `ensureAssetListed`, which only asks its own `poolRegistry`, the isolated pools registry. That returns false for the spoke comptroller, so the call reverts with `MarketNotExistInPool`. The whole `releaseFunds` call reverts, so while RiskFundConverter has a share in `distributionTargets`, no destination gets paid for the spoke pool.

Impact

Spoke pool income stays locked in ProtocolShareReserve until RiskFundConverter is upgraded. The risk fund also never receives any reserves for the spoke pool.

Code Snippet

ProtocolShareReserve.sol, `_isMarketRegistered` (checks every registry):

```
function _isMarketRegistered(address comptroller, address asset) internal view
↪  returns (bool) {
    address primary = poolRegistry;
    if (primary != address(0) &&
    ↪  IPoolRegistry(primary).getVTokenForAsset(comptroller, asset) != address(0))
    ↪  {
        return true;
    }

    uint256 length = additionalPoolRegistries.length;
    for (uint256 i; i < length; ) {
        if (IPoolRegistry(additionalPoolRegistries[i]).getVTokenForAsset(comptrolle
        ↪  r, asset) != address(0)) {
            return true;
        }

        unchecked {
            ++i;
        }
    }

    return false;
}
```

ProtocolShareReserve.sol, `_releaseFund`:

```
uint256[] memory totalTransferAmounts = new uint256[](totalSchemas);
for (uint256 i = 0; i < distributionTargets.length; ) {
    DistributionConfig memory _config = distributionTargets[i];

    uint256 transferAmount = (schemaBalances[uint256(_config.schema)] *
    ↪  _config.percentage) / MAX_PERCENT;
    totalTransferAmounts[uint256(_config.schema)] += transferAmount;

    if (transferAmount != 0) {
        IERC20Upgradeable(asset).safeTransfer(_config.destination, transferAmount);
        IIncomeDestination(_config.destination).updateAssetsState(comptroller,
        ↪  asset);

        emit AssetReleased(_config.destination, asset, _config.schema,
        ↪  _config.percentage, transferAmount);
    }
```

RiskFundConverter.sol, `_updateAssetsState` and `ensureAssetListed` (checks one registry):

```
function _updateAssetsState(address comptroller, address asset)
    internal
    override
```

```
    returns (uint256 balanceDifference)
{
    if (!ensureAssetListed(comptroller, asset)) revert
    ↪  MarketNotExistInPool(comptroller, asset);
```

```
function ensureAssetListed(address comptroller, address asset) internal view
↪  returns (bool isListed) {
    if (comptroller == CORE_POOL_COMPTROLLER) {
        isListed = isAssetListedInCore(asset);
    } else {
        isListed = IPoolRegistry(poolRegistry).getVTokenForAsset(comptroller,
        ↪  asset) != address(0);
    }
}
```

Tools Used

Manual Review

Recommendation

Make RiskFundConverter check every registry the PSR knows. One simple way is to store the PSR address and use its public `isMarketRegistered`:

```
function ensureAssetListed(address comptroller, address asset) internal view
↪  returns (bool isListed) {
    if (comptroller == CORE_POOL_COMPTROLLER) {
        isListed = isAssetListedInCore(asset);
    } else {
        isListed = IProtocolShareReserve(protocolShareReserve).isMarketRegistered(c
        ↪  omptroller, asset);
    }
}
```

Ship it together with, or before, adding `SpokePoolRegistry` to the PSR.

Discussion

Debugger022

Acknowledged, no code change. RiskFundConverter is not a ProtocolShareReserve distribution target on BSC mainnet, so this path isn't live. We read all 18 distribution rows on 2026-10-09, and none of them points to RiskFundConverter. We also simulated `updateAssetsState` from the ProtocolShareReserve with a comptroller that no registry knows. It succeeds on all 9 current destinations and reverts with `MarketNotExistInPool` only on RiskFundConverter.

The listing proposal doesn't change the distribution rows. If RiskFundConverter were added back as a target, `releaseFunds` for spoke pools would revert and the income would stay in the ProtocolShareReserve, with no loss. RiskFundConverter is also outside this audit's scope.

Issue L-1: `removePoolRegistry` NatSpec undersells what removing a live registry does: a routine removal freezes repays, withdrawals and liquidations [RESOLVED]

Source:
https://github.com/sherlock-audit2/2026-09-venus-protocol-sep-11th-2026/issues/6

Summary

The NatSpec on `removePoolRegistry` only says "Pools known only to it can no longer report income." In practice, removing a registry that still has live pools can freeze those pools' markets. This is an owner action. The only related warning, in the `addPoolRegistry` NatSpec (L179-182), covers repointing the primary registry, not removal. The function's own documentation tells governance the removal is harmless apart from income, while in reality it locks users out of repaying and withdrawing and stops liquidations. That mismatch is why this is filed as Low rather than Info.

Vulnerability Detail

After `removePoolRegistry(R)`, `updateAssetsState` reverts with `InvalidAddress` (L443) for any pool that only `R` knows about. Two vToken paths call it:

- Reserve reduction: `VToken.accrueInterest` calls `_reduceReservesFresh` once `reduceReservesBlockDelta` has passed (VToken.sol L886-893). For a market with non-zero reserves, that call reverts. The revert also rolls back the `reduceReservesBlockNumber` update, so every later `accrueInterest` hits the same branch. From then on, anything that accrues interest in that market reverts: mint, redeem, borrow, repay, liquidation, and `healAccount` / `liquidateAccount` through `_refreshMarkets`.
- Seizure: `_seize` calls `updateAssetsState` on every seizure, even when the protocol's share is zero (VToken.sol L1311-1318). So any liquidation or heal that seizes collateral in those pools reverts.

A market with zero reserves isn't frozen right away. It freezes once reserves build up and the delta passes, or as soon as someone tries to seize collateral in it.

The only warning in the contract is in `addPoolRegistry`'s NatSpec (L179-182), and it covers a different action, repointing the primary with `setPoolRegistry` ("repointing `poolRegistry` instead makes every pool of the old registry fail `updateAssetsState`"). `setPoolRegistry`'s own NatSpec (L165-169) has no warning at all. `removePoolRegistry`'s NatSpec (L203), the one governance reads before removing a registry, says the only effect is that pools "can no longer report income".

Impact

Removing a registry is a normal lifecycle step, for example sunsetting a product, so it isn't the owner doing something malicious or unexpected. An owner who follows the documentation for this function would reasonably expect income reporting to stop and nothing else. In reality, users in those pools can't repay or withdraw once reserves exist, and liquidations stop immediately. That can let bad debt build up before anyone notices the cause. The mismatch between the function's own documentation and its effect is what makes this a valid finding.

Code Snippet

- https://github.com/VenusProtocol/protocol-reserve/blob/a7b8f12de27657378d4173432852c70b000339e2/contracts/ProtocolReserve/ProtocolShareReserve.sol#L178-L228
- https://github.com/VenusProtocol/protocol-reserve/blob/a7b8f12de27657378d4173432852c70b000339e2/contracts/ProtocolReserve/ProtocolShareReserve.sol#L443
- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/VToken.sol#L886-L893 (context)
- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/VToken.sol#L1311-L1318 (context)

Tool Used

Manual Review

Recommendation

Update the `removePoolRegistry` NatSpec to match what `addPoolRegistry` already says, and spell out the result: once reserves exist, `accrueInterest` reverts for those markets, and every seizure reverts immediately. That blocks user actions and liquidations until the registry is added back.

Discussion

Debugger022

Valid. The NatSpec understated it. For markets that only the removed registry lists, `updateAssetsState` reverts, so every seizure in them reverts. Once a market's reserves are due to be swept to the PSR, every accrual reverts as well, which blocks every action that accrues first. They don't just stop reporting income.

We documented this on `removePoolRegistry` instead of blocking the removal. Blocking it would mean checking every pool and market of the registry, and there is no bounded way to do that: `PoolRegistry.getAllPools` is itself documented as too gas-intensive to call in a transaction. The NatSpec now names both paths, the protocol seize during liquidation and `accrueInterest` once `reduceReservesBlockDelta` passes, and notes that adding the registry back restores the markets.

Fixed in bc17c8a.

Issue L-2: `setMarketLiquidationIncentive` NatSpec tells governance to ”reset” a market by passing the pool value, but that pins it and it stops following the pool [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-venus-protocol-sep-11th-2026/issues/7

Summary

The NatSpec on `setMarketLiquidationIncentive` says there's no way to unset a market's incentive, and then tells governance: ”Pass that value explicitly to get the same effect.” It isn't the same effect. A market with no incentive of its own follows the pool-wide value, including every later change to it. A market given the pool value explicitly is pinned to that number forever. So a market ”reset” this way silently drifts away from the rest of the pool the next time the pool-wide incentive changes.

Vulnerability Detail

`_liquidationIncentive` (L1903-1906) only falls back to the pool value when the stored value is 0:

```
uint256 incentive = liquidationIncentives[vTokenCollateral];
return incentive != 0 ? incentive : _poolLiquidationIncentiveMantissa;
```

Zero can never be written back, because `setMarketLiquidationIncentive` requires `new >= 1e18 + protocolSeizeShare` (L1220). So after any market-specific value is set, the market is detached from the pool for good. The NatSpec's advice to pass the pool value explicitly doesn't restore the fallback. It just freezes today's pool value into the market.

Walkthrough:

1. The pool-wide incentive is 1.10.
2. During volatile conditions, governance raises market M to 1.20.
3. Conditions calm down. Following the NatSpec, governance ”resets” M with `setMarketLiquidationIncentive(M, 1.10e18)`.
4. Later, liquidations across the pool become unprofitable, so governance raises the pool-wide incentive to 1.15 with `setLiquidationIncentive`.
5. Every market without its own value moves to 1.15. M stays at 1.10. `NewLiquidationIncentive` has no market field, so nothing on-chain shows that M didn't move.

Liquidators now find M's collateral less attractive than everything else in the pool, so M's positions get liquidated later during the same stress that prompted the change. M's

lower incentive also gives it a larger `maxClearableDebt` term (L1825-1828), so a small account that should be healed can be routed to `liquidateAccount` instead.

Impact

Following the contract's own instructions leaves a market on a stale incentive without any signal. During stress, that market's liquidations lag the rest of the pool, and suppliers to that market carry the extra bad-debt risk. The fix is small, and the harm comes from wrong documentation rather than a governance mistake, so this is filed as Low.

Code Snippet

- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptroller.sol#L1187-L1227
- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptroller.sol#L1903-L1906

Tool Used

Manual Review

Recommendation

Either add a way to clear a market's incentive (for example a separate `clearMarketLiquidationIncentive(vToken)` that writes 0 on purpose, which still avoids the "mistaken zero" the NatSpec worries about), or fix the NatSpec to say that passing the pool value pins it and the market will no longer follow future `setLiquidationIncentive` changes.

Discussion

Debugger022

Valid. Passing the pool-wide value pins the market at that number instead of unsetting it. We corrected the NatSpec: it now says the market no longer follows later `setLiquidationIncentive` changes, so a pool-wide change has to update such markets too.

Fixed in 6beb6d2.

Issue L-3: The PSR says the second schema is ”liquidation income”, but vault protocol fees, flash-loan fees and wrapper rewards are split under it too [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-venus-protocol-sep-11th-2026/issues/8

Summary

The `Schema` NatSpec says: ”The first schema is for spread income. The second schema is for liquidation income.” Governance sets each schema's distribution percentages based on that. But `_getSchema` starts from `ADDITIONAL_REVENUE` and only switches for `SPREAD`. `IncomeType` now has six members, so four income types that aren't liquidations also land in the ”liquidation” schema: `ERC4626_WRAPPER_REWARDS`, `FLASHLOAN`, `INSTITUTIONAL_VAULT_PROTOCOL_FEE` and `INSTITUTIONAL_VAULT_LIQUIDATION`. `INSTITUTIONAL_VAULT_PROTOCOL_FEE` in particular is a fee on vault interest, not liquidation proceeds.

Vulnerability Detail

```
function _getSchema(IncomeType incomeType) internal view returns (Schema schema) {
    schema = Schema.ADDITIONAL_REVENUE;
    if (incomeType == IncomeType.SPREAD) {
        schema = Schema.PROTOCOL_RESERVES;
    }
}
```

The two schemas are split very differently on BSC mainnet today. I read the 18 live targets from the PSR (`0xCa01D5A9A248a830E9D93231e791B1afFed7c446`):

Destination	Schema 0 (spread)	Schema 1 (”liquidation”)
Prime buybacks (USDT + U)	40%	20%
Treasury buybacks	40%	60%
Risk fund buyback	20%	20%

So every unit of vault protocol fee or flash-loan fee sent to the PSR pays Prime holders half of what the same income would pay under the spread schema, and the treasury gets the difference. Governance tuned the schema-1 split while the only documentation said it covered liquidation income.

Impact

Protocol income from newer products is split under rules governance set for a different kind of income, which shifts value between Prime holders (users) and the treasury on every such payment. No funds are lost outright, and governance can re-tune the split once it knows, so this is filed as Low. The root is that the in-scope documentation of the schemas no longer matches what the code routes into them.

Code Snippet

- https://github.com/VenusProtocol/protocol-reserve/blob/a7b8f12de27657378d4173432852c70b000339e2/contracts/ProtocolReserve/ProtocolShareReserve.sol#L31-L37
- https://github.com/VenusProtocol/protocol-reserve/blob/a7b8f12de27657378d4173432852c70b000339e2/contracts/ProtocolReserve/ProtocolShareReserve.sol#L553-L559
- https://github.com/VenusProtocol/protocol-reserve/blob/a7b8f12de27657378d4173432852c70b000339e2/contracts/Interfaces/IProtocolShareReserve.sol#L6-L13

Tool Used

Manual Review

Recommendation

Map each `IncomeType` to a schema explicitly in `_getSchema` (for example, send `INSTITUTIONAL_VAULT_PROTOCOL_FEE` and `FLASHLOAN` to `PROTOCOL_RESERVES` if that's the intent) and revert on unknown types. At a minimum, fix the `Schema` NatSpec to list which income types each schema receives, so governance sets the percentages knowingly.

Discussion

Debugger022

Valid, the NatSpec was stale. We also confirmed your percentages on chain. We documented the routing instead of remapping it: the routing is intended, this upgrade leaves `_getSchema` unchanged, and remapping would change how live income is split, which needs a governance decision of its own. The `Schema` NatSpec now lists the income types each schema takes.

Fixed in 791c42c.

Issue L-4: The PSR's `maxLoopsLimit` can never be changed, and 18 of its 20 slots are already used [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-venus-protocol-sep-11th-2026/issues/9

Summary

`ProtocolShareReserve` sets `maxLoopsLimit` once, in `initialize`, and has no setter. `ConverterNetwork`, in the same repo, has a `setMaxLoopsLimit` guarded by an `InvalidMaxLoopsLimit` error. The PSR declares that same error (L19) but never uses it, which suggests a setter was meant to exist. On BSC mainnet the limit is 20 and there are already 18 distribution targets, so governance can add only two more destinations before `addOrUpdateDistributionConfigs` reverts with `MaxLoopsLimitExceeded`. The only way past that is an implementation upgrade. This upgrade also makes `addPoolRegistry` check the same fixed limit.

Vulnerability Detail

- `initialize` calls `_setMaxLoopsLimit(_loopsLimit)` (L162). Nothing else in the contract changes the limit.
- `addOrUpdateDistributionConfigs` ends with `_ensureMaxLoops(distributionTargets.length)` (L274).
- `addPoolRegistry` ends with `_ensureMaxLoops(additionalPoolRegistries.length)` (L197).
- Live values, read with `eth_call` against `0xCa01D5A9A248a830E9D93231e791B1afFed7c446`: `maxLoopsLimit() = 20`, `totalDistributions() = 18`.

A governance proposal that adds three new destinations (for example new buyback targets across both schemas) reverts. The only options are removing existing destinations first, which requires zeroing their percentages and re-balancing both schemas, or upgrading the PSR implementation just to raise a number.

Impact

Adding income destinations, which is routine for the PSR, is capped close to its current size, and getting past the cap needs a proxy upgrade instead of a parameter change. No funds are at risk. It's filed as Low because this upgrade touches the same loop limit (the new registry list uses it too) and was the natural place to add the setter that the unused error points to.

Code Snippet

- https://github.com/VenusProtocol/protocol-reserve/blob/a7b8f12de27657378d4173432852c70b000339e2/contracts/ProtocolReserve/ProtocolShareReserve.sol#L19
- https://github.com/VenusProtocol/protocol-reserve/blob/a7b8f12de27657378d4173432852c70b000339e2/contracts/ProtocolReserve/ProtocolShareReserve.sol#L159-L163
- https://github.com/VenusProtocol/protocol-reserve/blob/a7b8f12de27657378d4173432852c70b000339e2/contracts/ProtocolReserve/ProtocolShareReserve.sol#L189-L200
- https://github.com/VenusProtocol/protocol-reserve/blob/a7b8f12de27657378d4173432852c70b000339e2/contracts/ProtocolReserve/ProtocolShareReserve.sol#L234-L275

Tool Used

Manual Review

Recommendation

Add an owner-only `setMaxLoopsLimit(uint256)` that reverts with the already-declared `InvalidMaxLoopsLimit` on bad input, the same way `ConverterNetwork` does.

Discussion

Debugger022

Valid. Added `setMaxLoopsLimit` in 80975c4. It is owner-only like `setPoolRegistry`, and uses `InvalidMaxLoopsLimit` for the upper bound. No lower bound is needed: `_setMaxLoopsLimit` already requires the new limit to be above the current one, so the cap can only rise and the existing targets can never end up above it.

Issue L-5: `unlistMarket` doesn't require an empty market, so open positions vanish from every liquidity check and `healAccount` can write off a borrower's debt for free [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit2/2026-09-venus-protocol-sep-11th-2026/issues/10

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`unlistMarket` checks that every action is paused, both caps are zero and the collateral factor is zero. It never checks that the market has no supply or borrows left. Once `isListed` is false, `getAssetsIn` drops the market from every liquidity snapshot, heal and liquidation, and nothing can bring it back: `_setActionPaused` reverts `MarketNotListed` on an unlisted market, and `_addMarket` refuses to list it again.

Vulnerability Detail

Two things follow when positions are left behind:

- Debt in the unlisted market disappears. The borrower's other collateral, which was backing that debt, becomes free to borrow against or redeem. The debt can never be repaid, because REPAY has to be paused to unlist and can't be unpaused afterwards.
- Collateral in the unlisted market disappears while debt elsewhere stays. If that market was the account's only collateral, the account shows 0 collateral and a shortfall, so `maxClearableDebt` is 0 and the heal percentage is 0. Any caller can then run `healAccount` (the liquidation allowlist is off by default; `healAccount` checks the caller at L711), and `healBorrow(caller, user, 0)` skips `preRepayHook` and moves the whole debt into `badDebt` without the caller paying anything. If the account has other collateral, its collateral is only undervalued, and the normal heal or liquidation route applies.

Example for the second case: a user supplies 1,000 USDD, enters vUSDD and borrows 700 USDT. Governance winds vUSDD down and unlists it before the user redeems. Anyone then calls `healAccount(user)`: `getAssetsIn` returns only vUSDT, collateral is 0, the percentage is 0, and all 700 USDT becomes bad debt that USDT suppliers absorb.

Impact

Debt that can't be seen, liquidated or repaid, and a zero-cost write-off of a borrower's whole debt by anyone. It needs governance to unlist a market while positions are still open, which is easy to do with dust or inactive users left behind. The code is identical to upstream `Comptroller`, so this is Low.

Code Snippet

- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptroller.sol#L160-L221
- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptroller.sol#L1530-L1550
- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptroller.sol#L707-L760

Tool Used

Manual Review

Recommendation

Require `totalBorrows() == 0` and only dust supply in `unlistMarket`. A strict `totalSupply() == 0` would make unlisting practically impossible, because `PoolRegistry.addMarket` mints a permanent seed supply to `vTokenReceiver`. Alternatively, keep unlisted markets' positions in liquidity snapshots.

Discussion

Debugger022

Acknowledged, no code change.

- This logic is unchanged from upstream. Both the isolated-pools and the Core `unlistMarket` check only pauses, caps and the CF, not balances.
- Only an ACM-permissioned caller can reach this, and only by unlisting a market that still has open positions.
- A `totalBorrows() == 0` or dust-supply check doesn't fit within the contract size limit.
- Mitigation: before any unlisting, we will confirm that the market has zero borrows and that no entered account still holds debt elsewhere.

Issue L-6: The supply allowlist is only checked at mint, so restricted-market vTokens can be transferred or seized to non-allowlisted accounts, which can then borrow against them [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-venus-protocol-sep-11th-2026/issues/11

Summary

`isSupplyAllowlistEnabled` and `isAllowedSupplier` are only read in `preMintHook` (L336-338). `preTransferHook` only checks that the sender can redeem, and never looks at the receiver. `preSeizeHook` checks the liquidation allowlist, which is off by default, but not the supply allowlist. `enterMarkets` is unrestricted.

Vulnerability Detail

There's no borrow allowlist, so the spoke's borrow side is only restricted because a borrower needs collateral and collateral needs supply. If an allowlisted account transfers its vTokens (for example selling the position off-chain), the receiver can enter the market, use it as collateral and borrow:

1. Allowlisted account A mints vUSDT.
2. A transfers the vUSDT to non-allowlisted B. `preTransferHook` only checks that A can redeem.
3. B calls `enterMarkets([vUSDT])` and borrows USDC against it.

Seizure is another route: while the liquidation allowlist is off, any liquidator receives restricted vTokens. The deploy scripts describe the spoke as a pool whose supply, borrow and liquidation sides are restricted to known accounts (`deploy/024` L9-12).

Impact

Accounts the allowlist was meant to keep out can hold the market, earn from it and borrow against it. The in-scope NatSpec (SpokeComptroller.sol L1231-1233, SpokeComptrollerInterface.sol L298) does say only minting is gated, and it takes an allowlisted holder to transfer, so this is Low.

Code Snippet

- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptroller.sol#L326-L359

- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptroller.sol#L639-L655
- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptroller.sol#L573-L624

Tool Used

Manual Review

Recommendation

If the allowlist is meant to restrict who takes part, also check the receiver in `preTransferHook` and the liquidator in `preSeizeHook`, and optionally check it in `_addToMarket`. Otherwise, document that it only limits minting.

Discussion

Debugger022

Acknowledged, this is by design. The supply allowlist only exists so that the liquidity market is funded by the Hub alone, and the only account that will be on it is the Hub's Spoke YieldGroup. Borrowing is meant to be open to anyone with collateral.

- In our mainnet configuration the liquidity market, the one market with the allowlist on, has a collateral factor of 0. Its vTokens give no borrowing power to whoever holds them, so step 3 of the issue doesn't apply to it.
- The YieldGroup's adapter only mints and redeems, so the YieldGroup never borrows. With no debt, none of its vTokens can be seized. It can't transfer them either: `sweep` refuses a registered market, and a market can only be removed once the YieldGroup holds none of its vTokens.

We updated the comments to match in 738e8f2. The `setSupplyAllowlistEnabled` NatSpec now says transfers and seizures don't check the list, and the deploy and config comments no longer say borrowing is restricted.

Issue L-7: Unlisting any market breaks `RewardsDistributor.claimRewardToken(holder)` for every holder in the pool [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit2/2026-09-venus-protocol-sep-11th-2026/issues/12

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`unlistMarket` leaves the market in `allMarkets`, and `getAllMarkets()` keeps returning it. `RewardsDistributor.claimRewardToken(holder)` claims over `comptroller.getAllMarkets()` and requires every market to be listed (RewardsDistributor.sol L299-301, L343).

Vulnerability Detail

After the first unlist, the one-argument claim reverts with ”market must be listed” for every holder, including holders who never used the unlisted market. Holders can still call the two-argument version with their own list of markets, but rewards accrued in the unlisted market and not yet settled can't be claimed.

Impact

The standard claim path stops working for everyone in the pool after a normal wind-down step. There's a workaround, and the root is in the out-of-scope distributor, so this is Low.

Code Snippet

- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptroller.sol#L160-L221
- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptroller.sol#L1376-L1378

Tool Used

Manual Review

Recommendation

Remove the market from `allMarkets` when unlisting, or have the claim loop skip unlisted markets.

Discussion

Debugger022

Acknowledged, no code change. This is inherited unchanged from upstream: `unlistMarket` only clears `isListed` and `allMarkets` is append-only, so upstream isolated pools behave the same way. The spoke pool has no rewards distributor today, so this has no effect yet.

- If a distributor is added later, rewards in the listed markets stay claimable through the two-argument `claimRewardToken(holder, vTokens)`.
- Rewards a holder hasn't settled in an unlisted market can't be claimed, as you note. The claim checks only that a market is listed, not that it is unpaused. So if the pool has a distributor, before unlisting a market with rewards we will set its reward speeds to 0 and leave holders time to claim.
- Dropping unlisted markets from `allMarkets` doesn't fit within the contract size limit.
- Skipping unlisted markets in the claim loop would mean changing `RewardsDistributor`, the shared contract isolated pools use, which is outside the audit scope.

Issue L-8: Liquidity snapshots read markets that haven't accrued interest, so borderline-solvent accounts can be liquidated and accounts can slightly exceed the collateral factor [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit2/2026-09-venus-protocol-sep-11th-2026/issues/13

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

Only `healAccount` and `liquidateAccount` accrue interest on every entered market (`_refreshMarkets`) before taking a liquidity snapshot. Every other path reads stored borrow balances and exchange rates for some or all of the account's markets. `liquidateBorrow`, `borrow` and `redeem` accrue only the market being acted on (plus the collateral market in a liquidation). `transfer`, `transferFrom` and `exitMarket` accrue nothing at all, not even the market being transferred or exited.

Vulnerability Detail

What each path accrues before its snapshot:

Path	Accrued before the snapshot
`healAccount`, `liquidateAccount`	every entered market
`liquidateBorrow` -> `preLiquidateHook`	borrowed and collateral market
`borrow` -> `preBorrowHook`	the borrowed market
`redeem` -> `_checkRedeemAllowed`	the redeemed market
`transfer` / `transferFrom` -> `preTransferHook`	nothing (`VToken.transfer` has no `accrueInterest`, VTo
`exitMarket`	nothing

Stale data cuts both ways:

- Stale collateral is undervalued, because an idle market's exchange rate only rises when it accrues. In a liquidation this overstates the account's shortfall, so a liquidator can pick a route that leaves an idle collateral market un-accrued and liquidate an account that would be solvent after accrual, collecting the incentive.

- Stale debt is understated. In `borrow`, `redeem`, `transfer` and `exitMarket` this lets an account go slightly past the collateral factor.

Example: a market idle for 7 days at 10% borrow APR, 70% utilization and a 20% reserve factor understates collateral by about 0.107%. On $100k of collateral at LT 0.88 that's roughly $94 of phantom shortfall.

Impact

Accounts right at the margin can be liquidated while solvent, losing the incentive on up to the close factor of their debt. In the other direction, accounts can borrow, redeem, transfer or exit slightly past the collateral factor, which is stricter than the liquidation threshold, so they don't become instantly liquidatable. It's standard Compound behavior, identical upstream, and needs an idle market, so this is Low.

Code Snippet

- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptroller.sol#L505-L557
- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptroller.sol#L402-L460
- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptroller.sol#L262-L310
- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptroller.sol#L639-L655
- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptroller.sol#L1669-L1697
- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/VToken.sol#L153-L161

Tool Used

Manual Review

Recommendation

Accrue interest on every market in `getAssetsIn(account)` before each liquidity snapshot (in `preLiquidateHook`, `preBorrowHook` and `_checkRedeemAllowed`, which also covers `transfer` and `exitMarket`), the same way `healAccount` and `liquidateAccount` already do with `_refreshMarkets`.

Discussion

Debugger022

Acknowledged, no code change. This is inherited from upstream, where only `healAccount` accrues every entered market before the snapshot. The spoke adds the same refresh to `liquidateAccount` and leaves the other checks unchanged. As the issue notes, it needs an idle market and the error is small, while accruing every entered market would add an `accrueInterest()` call per entered market to every borrow, liquidation, and redeem, transfer or exit that runs the liquidity check. To keep idle markets from going stale, we'll add the spoke markets to the keeper that already calls `accrueInterest()` periodically on every Core pool market.

Issue L-9: `healAccount` ignores the LIQUIDATE pause, and a zero-repayment heal skips the REPAY pause check and borrow-reward settlement [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-venus-protocol-sep-11th-2026/issues/14

Summary

`healAccount` never checks `Action.LIQUIDATE`. When the repayment is 0, which is always the case for a borrower with no collateral, `VToken.healBorrow` skips `preRepayHook` (VToken.sol L556-558), so the REPAY pause check and the borrower's reward-index settlement don't run either.

Vulnerability Detail

Pausing LIQUIDATE and REPAY during an incident doesn't stop bad-debt write-offs. For example, governance pauses LIQUIDATE because of an oracle problem, a bad price briefly pushes an account's collateral below `minLiquidatableCollateral`, and anyone calls `healAccount`: the collateral is seized at the suspect price and the rest of the debt becomes bad debt despite the pause. On the zero-repayment path, the borrower's accrued borrow rewards are never settled.

Impact

The pause can't stop heals, and some borrow rewards are lost. It needs governance to be relying on the pause with the liquidation allowlist off, and the behavior is identical upstream, so this is Low.

Code Snippet

- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptroller.sol#L707-L760

Tool Used

Manual Review

Recommendation

Check `Action.LIQUIDATE` for each borrowed market in `healAccount`, and call `preRepayHook` in `healBorrow` even when the repayment is 0.

Discussion

Debugger022

Fixed the LIQUIDATE part in 67933bd. `healAccount` now checks the pause on every market the account borrows from, before healing that market's debt, so heals that repay nothing are blocked too.

We're leaving `healBorrow` as it is, since it's in the shared VToken and outside this scope. With nothing repaid there is no repayment for the REPAY pause to block, and the write-off itself is now gated by the LIQUIDATE pause. The `healAccount` NatSpec now lists every pause that stops a heal, including that the REPAY pause only applies where the heal repays a nonzero amount: 610faaf. Skipping the reward settlement means the defaulted borrower loses the borrow rewards that weren't credited to it yet. The spoke pool also has no rewards distributor.

Issue L-10: `maxLoopsLimit` of 100 allows a pool size where a borrower can enter every market and push liquidation past the per-transaction gas cap [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-venus-protocol-sep-11th-2026/issues/15

Summary

Every liquidation path loops over every market the borrower has entered, even markets where the borrower holds nothing. `enterMarkets` has no health check and no per-account limit, so a borrower who is already in shortfall can enter every listed market in one transaction, and can front-run a liquidator to do it. Each entered market adds a fixed cost to the liquidation. The pool's market count is capped only by `maxLoopsLimit`, which is 100. On a chain with a per-transaction gas cap, such as Ethereum's 16.78M under EIP-7825, a pool with enough markets lets a borrower make their own position impossible to liquidate.

No current pool is large enough for this. The problem is that the configured limit allows pools that are.

Vulnerability Detail

Entering a market only checks that ENTER_MARKET isn't paused, that the market is listed, and that the account isn't already a member (L1557-1579). There's no liquidity check and no per-account cap, and only the account itself can call `exitMarket`. The number of markets a pool can list is bounded by `maxLoopsLimit` (`_addMarket`, L1586-1597). It's set to 100 by `deploy/025`, and I confirmed on-chain that the testnet spoke and the Ethereum Core pool both use 100. It can only be raised.

Each liquidation path walks the full `getAssetsIn(borrower)` list:

- `liquidateBorrow` -> `preLiquidateHook`: `updatePrices(borrower)` plus a liquidation-threshold snapshot (a price read and a `getAccountSnapshot` per market).
- `liquidateAccount` (collateral <= `minLiquidatableCollateral`, $100 in the spoke config): `_refreshMarkets` (accrue interest and update the price per market, added by this fork), a snapshot, `updatePrices(borrower)` again in every order, and a final `getAccountSnapshot` per market.
- `healAccount`: `_refreshMarkets` and a snapshot per market, then a seize in every market where the account holds a balance.

Per-market costs (net of the 21k base cost), measured with `eth_estimateGas` against live Venus isolated pools:

Call	BSC (Stablecoins pool, 4 markets)	Ethereum (Core pool, 23 markets)
`oracle.updatePrice`	38k	55k
`oracle.getUnderlyingPrice`	139k	130k
`vToken.getAccountSnapshot`	38k	28k
`vToken.accrueInterest`	171k	107k

On Ethereum, each path needs roughly this many markets before it exceeds the 16,777,216 per-transaction cap:

Path	Gas per entered market	Stops fitting at about
`liquidateBorrow` (collateral > $100)	~212k	75 markets
`liquidateAccount` (collateral <= $100)	~400k	40 markets
`healAccount` (collateral <= $100)	~320k	50 markets

Today every pool is below those numbers. Ethereum's pools have 23, 9, 4 and 2 markets, and the spoke has 2. At 23 markets, liquidation costs roughly 5-9M, which fits. On BSC (70M block limit) even 100 markets fit.

Scenario once a pool is large enough:

1. The attacker borrows close to the collateral-factor limit.
2. The price moves and the account becomes liquidatable. The attacker calls `enterMarkets(allMarkets)`, or front-runs the first liquidation with it. No deposits in those markets are needed.
3. Every `liquidateBorrow`, `liquidateAccount` and `healAccount` call for the account needs more gas than a transaction may use, so none can be mined. The debt stays in `totalBorrows` and suppliers eventually absorb it.

A related variant, a dust borrow in every market, makes `liquidateAccount` grow with the square of the market count, because every order re-runs `updatePrices` over all markets. It can be defused, though: `repayBorrowBehalf(borrower, type(uint256).max)` is open to anyone, so a liquidator can clear the dust borrows first in separate transactions.

The spoke makes the batch path heavier than upstream: upstream `liquidateAccount` doesn't refresh markets first, and the spoke's added `_refreshMarkets` (L784-789) roughly doubles its per-market cost.

Impact

For a pool with roughly 40-75 or more listed markets, which `maxLoopsLimit = 100` permits, deployed on a chain with a per-transaction gas cap, a borrower can make an underwater position permanently unliquidatable. On the large-position path there's no limit on the position's size. No existing pool is that large, it needs governance to list that many markets, and the spoke is currently configured only for BSC, where every path still fits. So this is Low: the limit the contract accepts doesn't match what liquidation can actually handle.

Code Snippet

- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptroller.sol#L109-L121
- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptroller.sol#L1557-L1597
- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptroller.sol#L505-L557
- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptroller.sol#L707-L760
- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptroller.sol#L780-L841

Tool Used

Manual Review

Recommendation

- Set `maxLoopsLimit` per chain from measured liquidation gas, so `liquidateAccount` and `healAccount` stay well under the per-transaction cap at the maximum market count.
- Or cap how many markets one account can enter, or skip markets where the account has neither a balance nor a borrow in the liquidation loops.
- Optionally refuse `enterMarkets` while the account is in shortfall. A borrower can still add collateral by supplying.

Discussion

Debugger022

Fixed in 6b314fa. In Phase 1 the spoke pool is planned to have no more than 10 markets, so the spoke comptroller is now initialized with `maxLoopsLimit = 50`. `setMaxLoopsLimit` can only raise it, so we will measure liquidation gas again before any increase.

Issue L-11: `healAccount` and every liquidation path pay the protocol seize share out of the collateral market's cash, so a fully borrowed collateral market can't be seized, and the supply allowlist can stop liquidators from restoring cash [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit2/2026-09-venus-protocol-sep-11th-2026/issues/16

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

When collateral is seized, `VToken._seize` converts the protocol's share of the seized vTokens into underlying and transfers it to the ProtocolShareReserve from the collateral market's own cash. `_doTransferOut` does `internalCash -= amount`, which reverts on underflow. So whenever a collateral market has less free cash than `protocolSeizeShare / liquidationIncentive` of the seized value, every seize of that market reverts: `liquidateBorrow`, every `liquidateAccount` order that seizes it, and the seize loop in `healAccount`, which seizes the borrower's entire balance in every market. A market at 100% utilization can't be used to liquidate anyone.

The usual way out is for the liquidator to mint a little into the collateral market first, which restores cash. The spoke's new supply allowlist blocks that for anyone who isn't an allowed supplier, and liquidation and supply allowlists are separate lists.

Vulnerability Detail

`VToken._seize` (L1278-1326):

```
uint256 protocolSeizeTokens = div_(numerator, Exp({ mantissa:
↪  liquidationIncentiveMantissa }));
uint256 liquidatorSeizeTokens = seizeTokens - protocolSeizeTokens;
Exp memory exchangeRate = Exp({ mantissa: _exchangeRateStored() });
uint256 protocolSeizeAmount = mul_ScalarTruncate(exchangeRate, protocolSeizeTokens);
...
totalSupply = totalSupply - protocolSeizeTokens;
...
_doTransferOut(protocolShareReserve, protocolSeizeAmount);   // internalCash -=
↪  amount, reverts if cash is short
```

`healAccount` (L745-759) seizes `tokens` (the borrower's full balance) in every market before repaying, so it needs `protocolSeizeAmount = share / incentive * value(tokens)` of free

cash in each collateral market. With the default 5% share and a 1.10 incentive that is about 4.5% of the account's collateral value in that market, in cash.

How a borrower can put themselves in this position:

1. The attacker supplies collateral in market A and borrows in market B, close to the limit.
2. From a second account (with collateral elsewhere) the attacker borrows all of A's free cash. Borrowing is only limited by `borrowCaps` (L432-434), so this works whenever the cap is at least A's supply. Utilization goes to 100%, and the second account pays the jump rate for as long as it holds the loan.
3. The first account becomes liquidatable. Every `liquidateBorrow(borrower, repay, vA)`, every `liquidateAccount` order with `vTokenCollateral == vA`, and `healAccount(borrower)` revert inside `_doTransferOut`, because A has no cash to pay the protocol's share.
4. A liquidator can normally fix this by minting into A. If A's supply allowlist is on (`preMintHook`, L336-338) and the liquidator isn't an allowed supplier, or A's supply cap is reached, they can't. Allowed suppliers and repayers of A still can, and governance can set `protocolSeizeShare` to 0 for A, but the position stays open until one of them acts.

Note the mismatch: `liquidateAccount` and `healAccount` seize the collateral in vTokens, which never needs cash, and the only cash movement is the protocol's fee. Paying that fee in vTokens (leaving the protocol's share as vTokens held by the PSR or the reserve) would remove the cash dependency entirely.

Impact

A borrower who can push their collateral market to full utilization, at the cost of paying the jump rate from a second account, can't be liquidated or healed through that market until someone else adds cash to it. `healAccount` is the path meant for accounts that are already underwater, and it's the one most affected because it seizes 100% of every balance. With the supply allowlist on, the set of people who can restore cash shrinks to the allowlisted suppliers. The cash transfer in `_seize` is upstream code and the fee design is deliberate, so this is Low: the in-scope batch paths inherit a liveness dependency on cash that they don't need.

Code Snippet

- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptroller.sol#L745-L759
- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptroller.sol#L336-L338
- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/VToken.sol#L1278-L1326

- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/VToken.sol#L1491-L1495

Tool Used

Manual Review

Recommendation

- In `_seize`, when the collateral market doesn't have `protocolSeizeAmount` of free cash, credit the protocol's share as vTokens (transfer `protocolSeizeTokens` to the ProtocolShareReserve instead of burning them and paying underlying), or add the shortfall to `totalReserves` and let the normal reserve reduction pay it out later.
- Or exempt liquidators from the supply allowlist for the amount needed to cover the seize share, so the existing workaround keeps working on allowlisted markets.

Discussion

Debugger022

Acknowledged, no code change. The protocol-share transfer is in `VToken._seize`, the shared VToken that every isolated pool uses. The spoke markets run that contract unchanged.

Your steps 1 and 2 need one market that the borrower supplies as collateral and that a second account can borrow in full. In the planned mainnet configuration, no spoke market is both:

- The collateral markets have a borrow cap of 0, with BORROW paused as a backup, so no one can borrow their cash. With no borrows, a market's cash always covers its suppliers' balances, so it always has the cash for the protocol's share.
- The liquidity market, the only borrowable one, has a collateral factor of 0. Its supply allowlist is on, so a borrower can't supply to it.

The supply allowlist is enabled only on the liquidity market, so it never stops a liquidator from adding cash to a collateral market.

Issue L-12: `healAccount` NatSpec says spoke bad debt can be auctioned, but for spoke pools it can't [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-venus-protocol-sep-11th-2026/issues/17

Summary

The contract-level NatSpec says the debt `healAccount` forgives is "recorded as `badDebt` for each market, which can then be auctioned off for the risk reserves of the pool". With the current deployment, that isn't true for spoke pools. The code in `SpokeComptroller` is fine. This is a documentation mismatch, and the actual cause lives in files outside the scope.

Vulnerability Detail

- Only a vToken's `shortfall` address can lower its `badDebt`, via `VToken.badDebtRecovered`.
- `deploy/028-deploy-spoke-vtokens.ts` (L68-72) points every spoke market at the chain's existing `Shortfall`.
- That `Shortfall` finds pools through a single `poolRegistry`, the isolated-pools one (`Shortfall.sol` L403-405). Spoke pools are registered only in `SpokePoolRegistry`, on purpose.
- So `startAuction(spokeComptroller)` reverts, and spoke `badDebt` never goes down.

What follows from that, all outside the scope files:

- `preBorrowHook` keeps counting the bad debt against the borrow cap (L431-432). That's the same behavior as upstream `Comptroller`.
- The exchange rate keeps it in the numerator.
- The Hub adapter's comments (`AdapterSpokeV1.sol` L42-43) assume an auction will eventually lower it.

`Shortfall.sol` and the deploy script aren't in scope, so this is filed as informational against the one in-scope line that promises the auction.

Impact

No funds are lost directly. Governance and integrators (the Hub adapter's comments assume an auction will eventually lower `badDebt`) are told a recovery path exists for spoke

bad debt when it doesn't, so that debt is never auctioned, keeps counting against the borrow cap, and stays in the exchange rate.

The practical impact is also smaller than it first looks. On BSC mainnet, the live `Shortfall` (`0xf37530A8a810Fcb501AA0Ecd0B0699388F0F2209`) uses `RiskFundV2` (`0xdF31a28D68A2AB381D42b380649Ead7ae2A76E42`) as its risk fund. `RiskFundV2.getPoolsBaseAssetReserves` always returns 0 (RiskFundV2.sol L127-130). I checked this on-chain: it returns 0 for a live isolated pool, even though the contract holds about 3.44M USDT. So an auction would start as `LARGE_POOL_DEBT` with nothing seized from the risk fund, for any pool, isolated or spoke. Spoke pools don't lose risk-fund coverage that isolated pools have. What they lose is the auction as a way for someone (for example the treasury) to repay bad debt and call `badDebtRecovered`.

Code Snippet

- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptroller.sol#L44-L50
- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptroller.sol#L427-L436
- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Shortfall/Shortfall.sol#L403-L405 (out of scope, context)
- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/deploy/028-deploy-spoke-vtokens.ts#L68-L72 (out of scope, context)

Tool Used

Manual Review

Recommendation

Update the NatSpec so it doesn't promise an auction for spoke pools. Separately, and outside this audit's scope, the team may want to deploy a `Shortfall` pointed at `SpokePoolRegistry` (or make `Shortfall` look up multiple registries like the PSR now does) if spoke bad debt is supposed to be recoverable.

Discussion

Debugger022

Fixed in 42fc51c. The NatSpec no longer says the bad debt can be auctioned. It now says `Shortfall` can't auction spoke bad debt, because `Shortfall` only finds pools in the pool registry it is configured with, and that registry doesn't hold spoke pools. The deploy

scripts carry the same caveat: `024` next to the ProtocolShareReserve note (281a68e), and `028`, which wires the spoke markets to `Shortfall` (0534dc5).

Recovery doesn't depend on an auction. The listing VIP points every spoke market's `shortfall` at the Normal Timelock, so governance can fund a market and call `badDebtRecovered` to lower its `badDebt`. The Hub adapter's comments (outside this scope) now refer to the market's `shortfall` calling `badDebtRecovered`, not to an auction.

Issue L-13: Nothing keeps `liquidationThreshold * incentive <= 1` for a market, so a later threshold change can route solvent accounts into heal and make self-heal farming profitable [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-venus-protocol-sep-11th-2026/issues/18

Summary

In the spoke, each market's liquidation threshold (LT) and its own liquidation incentive (LI) are set in two different functions:

- `setCollateralFactor` checks `CF <= 0.95`, `LT <= 1` and `LT >= CF`. It never looks at the market's incentive.
- `setMarketLiquidationIncentive` (new in this fork) only checks `LI >= 1e18 + protocolSeizeShare`. It never looks at the market's threshold.

For small accounts, the incentive decides the route between `liquidateAccount` and `healAccount` through `maxClearableDebt = sum(collateral_i / LI_i)`. If `LT * LI > 1` for a market, a small account using it can be healed the moment it reaches shortfall, while its collateral still covers its debt, and the difference is written off as bad debt. If `LT * LI > 1 + share`, a borrower can heal their own accounts from a second address for a profit.

Neither setter can stop this, because the unsafe combination only exists across the two. A pair of values that is safe when set can become unsafe after a routine change to the other value.

Vulnerability Detail

Sequence, with numbers:

1. Market M is illiquid. Governance sets its own LI to 1.30 while CF / LT are 0.60 / 0.70. LT * LI = 0.91, which is safe.
2. Later, governance raises CF / LT to 0.80 / 0.85 through `setCollateralFactor`. Both checks pass. LT * LI is now 1.105, and nothing flags it.
3. A borrower supplies 100 USD of M and borrows 80 USD, right at the CF limit.
4. M's price drops 6.5%, so the collateral is worth 93.5. The account is in shortfall (0.85 * 93.5 = 79.48 < 80) but still solvent (93.5 > 80).
5. `maxClearableDebt` = 93.5 / 1.30 = 71.92 <= 80, so `liquidateAccount` reverts (L801) and `healAccount` is the only path (L736).

6. The healer pays 71.92 and gets 93.5 * (1 - 0.05 / 1.30) = 89.90 of collateral. The protocol gets 3.60, and 80 - 71.92 = 8.08 is written off as bad debt, even though the collateral covered the loan.
7. If the borrower is also the healer, they finish with 97.98 instead of the 93.5 they'd have after repaying and withdrawing. That's 4.48 per account, paid by suppliers. Any user can repeat it across many accounts kept below `minLiquidatableCollatera l`, so the gain grows with the number of accounts.

For comparison, with LI 1.10 the same account has `maxClearableDebt` = 85 > 80, goes through `liquidateAccount`, and nothing is written off.

The file header documents the routing formula, and that part isn't in question. But it describes heal as the path where "`badDebt` would be created". It doesn't say an account whose collateral still covers its debt will be sent there and have part of that debt written off. That only happens when `LT * LI > 1`, a combination of two parameters that the code sets in separate functions and never checks together.

Upstream has one pool-wide incentive and no such check either. The fork adds a per-market incentive setter meant for giving riskier collateral a bigger discount, which is exactly the kind of market where LI is high. It validates the lower side (so liquidators stay whole) but not the side that protects suppliers. The user-side exploit (self-heal farming) doesn't need any mistake. It needs a pair of individually valid settings, reached in the order shown above.

Impact

Debt on solvent small accounts gets written off, and once `LT * LI > 1 + share` holds for any market, any user can farm the difference across many accounts at suppliers' expense. "Solvent" here means collateral worth more than debt. The contract treats such an account as short of the liquidation bonus, which is how heal is defined in the file header, and the account also has to be below `minLiquidatableCollateral` to reach `healA ccount`. Upstream `setLiquidationIncentive` (Comptroller.sol L1073-1074) has the same gap between the threshold and incentive setters. Since it needs governance to put the two parameters in that range and matches the documented heal rule, it's filed as Info. Checking `LT * LI <= 1` in both setters is still a cheap safeguard.

Code Snippet

- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptroller.sol#L1206-L1227
- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptroller.sol#L879-L923
- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptroller.sol#L734-L758

- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptroller.sol#L1817-L1829

Tool Used

Manual Review

Recommendation

Enforce `liquidationThresholdMantissa * effectiveIncentive <= 1e18` in both `setMarketLiquidationIncentive` and `setCollateralFactor`, where `effectiveIncentive` is the pool-wide value if the market has no incentive of its own. Checking it in both places covers either order of changes. A sensible upper bound on the incentive is a cheap extra.

Discussion

Debugger022

Partially fixed in 82fc9f5. `setCollateralFactor` and `setMarketLiquidationIncentive` now revert with `UnsafeLiquidationParams` when LT × LI is 1 or more, the same bound the Core pool enforces. `setCollateralFactor` checks the market's effective incentive, so step 2 of your sequence now reverts: raising LT to 0.85 while the market's incentive is 1.30. This replaces our inline reply on PR #3, which said the check didn't fit under the size limit.

`setLiquidationIncentive` does not recheck the markets that fall back to the pool value, because that loop doesn't fit under the contract size limit. Its NatSpec says so. The listing proposal gives every collateral market its own incentive, so the pool value applies only to markets with a liquidation threshold of 0.

Issue L-14: The pool-wide incentive floor only protects markets on the default seize share, so a raised share or a changed VToken default can make liquidations unprofitable [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-venus-protocol-sep-11th-2026/issues/19

Summary

A market that doesn't have its own incentive uses the pool-wide one. For liquidating that market to pay, the pool incentive has to stay at or above `1e18 + protocolSeizeShareMantissa` of the market. `setLiquidationIncentive` only enforces the fixed floor `MIN_POOL_LIQUIDATION_INCENTIVE_MANTISSA = 1.05e18`, which is `1e18` plus VToken's default 5% share, typed in by hand. So the floor protects a market only while its share equals today's default. There are two ways a market can end up with a higher share than the floor covers:

1. Governance raises the market's share with `VToken.setProtocolSeizeShare` while it still uses the pool incentive, and later lowers the pool incentive.
2. A VToken beacon upgrade raises the default share, and markets listed after it start with that higher share.

In both cases the pool incentive can sit at a value where a liquidator gets back less than they repaid, and nobody liquidates, heals or batch-liquidates accounts using that collateral.

The NatSpec states the rule: the pool value "has to stay at or above `1e18 + protocolSeizeShareMantissa` of every market that has no incentive of its own", and "a market whose share is raised above the default still needs an incentive of its own". It says only the default floor is enforced because the shares "are only readable market by market".

Vulnerability Detail

- `VToken.setProtocolSeizeShare` reads `liquidationIncentiveMantissa()`. For a market with no incentive of its own that is the pool value, and any share up to that value minus 1e18 is accepted (VToken.sol L472-482). It doesn't require the market to have its own incentive first, so "a raised share needs its own incentive" isn't enforced anywhere.
- `setLiquidationIncentive` only checks `new >= 1.05e18` (SpokeComptroller.sol L951-953).
- In `VToken._seize` the protocol takes `seizeTokens * share / incentive`, so the liquidator keeps `repay * (incentive - share)` worth of collateral (VToken.sol

L1292-1296).

Case 1, a raised share:

1. The pool incentive is 1.10. Market M has no incentive of its own.
2. Governance raises M's share to 0.08. Allowed, because 1.08 <= 1.10.
3. Later, governance lowers the pool incentive to 1.06. That passes the 1.05 floor.
4. A liquidator repays 100 USD and seizes 106 USD of M. The protocol takes 106 * 0.08 / 1.06 = 8, so the liquidator keeps 98 and loses 2.
5. Healing loses money too: the healer pays C / 1.06 = 0.943 * C and gets C * (1 - 0.08 / 1.06) = 0.925 * C.

Nothing catches this afterwards, because `setProtocolSeizeShare` only checks when the share itself changes.

Case 2, a changed default: spoke vTokens are `BeaconProxy` instances behind `SpokeVToken Beacon` (deploy/027, 028), so one upgrade changes `VToken` for every market. If a new implementation raises the default share to 7%, markets listed afterwards start at 7% while the floor still allows a pool incentive of 1.05. A liquidator repays 100, seizes 105, the protocol takes 105 * 0.07 / 1.05 = 7, and the liquidator keeps 98.

Checking every market is cheap, since `allMarkets` is capped by `maxLoopsLimit` (`_addMarke t` calls `_ensureMaxLoops`).

Impact

Accounts with collateral in an affected market can't be profitably liquidated through any of the three paths until governance fixes the share or gives the market its own incentive. If prices fall meanwhile, those positions turn into bad debt that suppliers pay for. Case 1 needs two governance actions, and case 2 needs a future VToken upgrade, and the code acknowledges the gap (`SpokeComptrollerStorage.sol` L135-137 calls the floor "a backstop for that case, not a full guarantee"). So it's filed as Info.

Code Snippet

- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptroller.sol#L925-L963
- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptroller.sol#L1586-L1597
- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptrollerStorage.sol#L131-L138
- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/VToken.sol#L472-L482

- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/VToken.sol#L1292-L1296

Tool Used

Manual Review

Recommendation

Check the new pool value against every market that actually uses it:

```
uint256 count = allMarkets.length;
for (uint256 i; i < count; ++i) {
    address m = address(allMarkets[i]);
    if (markets[m].isListed && liquidationIncentives[m] == 0) {
        if (newLiquidationIncentiveMantissa < MANTISSA_ONE +
        ↪  VToken(m).protocolSeizeShareMantissa()) {
            revert InvalidLiquidationIncentive();
        }
    }
}
```

`allMarkets` is already limited by `maxLoopsLimit`, so the loop is safe. This also covers case 2, but it's still worth deriving `MIN_POOL_LIQUIDATION_INCENTIVE_MANTISSA` from a constant shared with `VToken`, or adding a test that fails if the two differ.

Discussion

Debugger022

Case 1, a raised share: acknowledged, no code change.

- Only governance can trigger this, and it takes two separate governance actions: raising a market's seize share, then lowering the pool incentive.
- The suggested loop doesn't fit within the contract size limit.
- Mitigation: every VIP that lowers the pool incentive will keep it at or above `1e18 + protocolSeizeShareMantissa` for each market that has no incentive of its own. Any VIP that raises a market's share above the default will also set that market's own incentive.

Case 2, a changed default: fixed in ccf9502 by pinning the floor in a test. The test reads the seize share from a real, unmodified VToken and checks that the pool-wide floor is exactly `1e18 + share`, so changing VToken's default fails CI until the constant is updated. It also checks that a liquidation at that floor pays the liquidator back exactly what it repaid.

Issue L-15: No fault isolation between pool registries: once there are several extra registries, one that reverts blocks the pools of the registries after it [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit2/2026-09-venus-protocol-sep-11th-2026/issues/20

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`_isMarketRegistered` asks the registries one by one: first the primary, then each entry in `additionalPoolRegistries` in array order. There's no `try/catch`, so if any registry reverts, the whole lookup reverts. A pool is only found after every registry before it has answered.

So when one registry fails, it doesn't just break its own pools. It breaks the pools of every registry that comes after it in the list, even though those pools have nothing to do with it. `updateAssetsState` then reverts for them, which freezes seizures, and markets with reserves also freeze once reserves are due.

Vulnerability Detail

```
function _isMarketRegistered(address comptroller, address asset) internal view
↪  returns (bool) {
    address primary = poolRegistry;
    if (primary != address(0) &&
    ↪  IPoolRegistry(primary).getVTokenForAsset(comptroller, asset) != address(0))
    ↪  {
        return true;
    }
    uint256 length = additionalPoolRegistries.length;
    for (uint256 i; i < length; ) {
        if (IPoolRegistry(additionalPoolRegistries[i]).getVTokenForAsset(comptrolle
        ↪  r, asset) != address(0)) {
            return true;
        }
        ...
    }
    return false;
}
```

The failure doesn't need a bad address to be entered. A registry that was correct when added can start reverting later. A proxy upgrade to an implementation without `getVToke`

`nForAsset` would do it, and so would a proxy that is paused or deliberately broken while the product it served is wound down. Either way, the pools of every registry checked after it (spoke pools, for example) start failing `updateAssetsState`:

- Every seizure in those pools reverts, because `_seize` calls `updateAssetsState` unconditionally (VToken.sol L1311-1318). That includes liquidations and heals.
- Markets with reserves revert inside `accrueInterest` once `reduceReservesBlockDelta` passes (VToken.sol L886-893), which also blocks mint, redeem, borrow and repay (see L-01).

`removePoolRegistry` swaps the removed entry with the last one and pops it (L214-217). That changes the order, so which pools are exposed to a given registry can change after an unrelated removal.

It's true that `setPoolRegistry` also accepts any non-zero address, and a bad primary breaks every non-core pool. That part is owner configuration, and I'm not claiming it. The problem here is new with this change.

With a single registry, a failure only affects that registry's own pools. With the list, one registry's failure spreads to pools served by other registries, and the order that decides who gets hit isn't stable. Registries are separate products with separate upgrade paths and ACM permissions (the deploy/024 comments give that as the reason spoke pools got their own registry). A lookup that lets one product's registry freeze another product's markets undoes that separation. The owner can fix it by removing the broken entry, but until a governance proposal gets through, liquidations in the unrelated pools are blocked.

Impact

In the deployment this change is written for, the isolated-pools registry is the primary and `SpokePoolRegistry` is the only additional entry. There, a failing spoke registry only affects spoke pools, because the primary resolves isolated pools before the list is checked. A real `PoolRegistry` also can't pause, and `getVTokenForAsset` is a plain mapping read that returns `address(0)` instead of reverting (PoolRegistry.sol L263-265). So a revert needs governance to replace a registry with something that isn't one, and the cross-registry freeze only becomes possible once a second additional registry is added. That makes this defense in depth for future registries, filed as Info.

Code Snippet

- https://github.com/VenusProtocol/protocol-reserve/blob/a7b8f12de27657378d4173432852c70b000339e2/contracts/ProtocolReserve/ProtocolShareReserve.sol#L189-L200
- https://github.com/VenusProtocol/protocol-reserve/blob/a7b8f12de27657378d4173432852c70b000339e2/contracts/ProtocolReserve/ProtocolShareReserve.sol#L209-L228

- https://github.com/VenusProtocol/protocol-reserve/blob/a7b8f12de27657378d4173432852c70b000339e2/contracts/ProtocolReserve/ProtocolShareReserve.sol#L528-L546

Tool Used

Manual Review

Recommendation

Wrap each additional-registry lookup in `_isMarketRegistered` in `try/catch` and skip any registry that reverts, so one registry's failure can't affect pools served by the others. As a cheap extra, have `addPoolRegistry` call `getVTokenForAsset(address(0), address(0))` once before storing the address.

Discussion

Debugger022

Acknowledged, leaving it for now. With the listing proposal, the isolated-pools registry stays primary and `SpokePoolRegistry` is the only additional registry. As the issue notes, a failure there only affects spoke pools, and the cross-registry case needs a second additional registry.

- `PoolRegistry.getVTokenForAsset` is a single mapping read with no require, no loop and no external call, so we could not find a path for it to revert.
- try/catch would catch a registry that reverts, but not a call to an address with no code. We tested this: that revert is raised in the calling contract, outside the try.
- A probe in `addPoolRegistry` would catch a wrong address at add time, but not a registry that starts reverting later, which is the case this issue describes.

Issue L-16: `setCollateralFactor` checks the spot oracle, but CF checks use the bounded oracle [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit2/2026-09-venus-protocol-sep-11th-2026/issues/21

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`setCollateralFactor` only refuses a non-zero collateral factor if the spot oracle (`oracle.getUnderlyingPrice`) returns 0. In the spoke, every check that uses the collateral factor gets its prices from `deviationBoundedOracle.getBoundedPricesView` instead (L1870-1876). So a market can be given a non-zero CF while `deviationBoundedOracle` is still unset.

Vulnerability Detail

```
// If collateral factor != 0, fail if price == 0
if (newCollateralFactorMantissa != 0 && oracle.getUnderlyingPrice(address(vToken))
↪  == 0) {
    revert PriceError(address(vToken));
}
```

If CF is set before `setDeviationBoundedOracle` is called, members of that market can't borrow, redeem, transfer or `exitMarket`. Each of those runs the CF liquidity check, which calls the zero address and reverts.

Once the bounded oracle is set, it returns spot for any asset it has no configuration for. So a zero bounded price can only happen when spot is also zero, and the existing check already catches that. The unset case is the only gap.

Impact

With a wrong setup order, members of a market are temporarily blocked from borrowing, redeeming, transferring and exiting until governance sets the bounded oracle. The `setDeviationBoundedOracle` NatSpec documents this fail-closed behavior, so this is Info.

Code Snippet

- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptroller.sol#L907-L910

- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptroller.sol#L1861-L1877

Tool Used

Manual Review

Recommendation

When `newCollateralFactorMantissa != 0`, also require that `deviationBoundedOracle` is set.

Discussion

Debugger022

Acknowledged, no code change.

- Only governance can trigger this, and only by listing markets before setting the bounded oracle. `setDeviationBoundedOracle` rejects the zero address, so once set it can't be unset.
- The bounded oracle is required for the whole pool, not just for markets with a non-zero CF. A borrow, and a redeem by a member of the market, query it for every market the account has entered, so they revert while it is unset even when every CF is 0. A check in `setCollateralFactor` would not cover that.
- This fail-closed behavior is intentional. No funds are at risk: deposits, repayments and liquidations keep working, non-members can still withdraw, and normal operation resumes once the oracle is set.
- Mitigation: the listing VIP sets the bounded oracle before the pool is registered and before any market is listed.

Issue L-17: PSR distribution config changes can mis-route or silently strand income: new splits apply to unreleased income, a schema can total 0%, and a failed removal says nothing [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-venus-protocol-sep-11th-2026/issues/22

Summary

Three gaps in how `ProtocolShareReserve` handles its distribution config, all in the same functions:

1. A config change applies to income that's booked but not yet released, so income earned under the old split is paid out under the new one.
2. `_ensurePercentages` accepts a schema whose targets total 0%, but `updateAssetsState` keeps crediting that schema, and `releaseFunds` returns success without moving it.
3. `removeDistributionConfig` returns normally with no event when nothing matches, including when the target's percentage isn't 0 yet.

Vulnerability Detail

Unreleased income. The PSR books income per pool, asset and schema, and only splits it when someone calls `releaseFunds`, using the percentages in force at that moment. `addOrUpdateDistributionConfigs` (L234) and `removeDistributionConfig` (L282) change the percentages without releasing first. With the live BSC schema-0 split (Prime buybacks 40%, treasury buybacks 40%, risk fund buyback 20%):

1. A pool has 100k USDT of spread income booked and not released.
2. A proposal cuts Prime to 20% and raises the treasury to 60%.
3. After it executes, someone calls `releaseFunds`.
4. Prime gets 20k and the treasury 60k, although the income was earned while Prime was entitled to 40k.

The amount moved is always the unreleased balance times the percentage change.

A schema at 0%. `_ensurePercentages` (L577-580) accepts a per-schema total of exactly `MAX_PERCENT` or exactly `0`. A schema at 0 still receives income through `updateAssetsState`, but `_releaseFund` has no target to pay, so that income sits in the reserve until a target is added, and `releaseFunds` still succeeds, which hides it.

A silent removal. `removeDistributionConfig` (L282-313) only removes an entry whose percentage is already 0. If no such entry exists, it emits nothing and returns normally, so a proposal can look like it removed a target when it didn't.

Impact

Income moves between protocol destinations (for example from Prime holders to the treasury), can pile up with no release path, and a failed removal isn't visible on-chain. Nothing is lost outright, each case needs a governance action, and `releaseFunds` is permissionless, so balances can be flushed before a change. The code is pre-existing, so this is Info.

Code Snippet

- https://github.com/VenusProtocol/protocol-reserve/blob/a7b8f12de27657378d4173432852c70b000339e2/contracts/ProtocolReserve/ProtocolShareReserve.sol#L234-L275
- https://github.com/VenusProtocol/protocol-reserve/blob/a7b8f12de27657378d4173432852c70b000339e2/contracts/ProtocolReserve/ProtocolShareReserve.sol#L282-L313
- https://github.com/VenusProtocol/protocol-reserve/blob/a7b8f12de27657378d4173432852c70b000339e2/contracts/ProtocolReserve/ProtocolShareReserve.sol#L564-L585

Tool Used

Manual Review

Recommendation

- Release income for the affected assets inside the config functions before applying the new split, or at least document that `releaseFunds` has to be called first for every comptroller and asset.
- Require every schema that can receive income to total 100%, or don't credit a schema that has no targets.
- Revert in `removeDistributionConfig` when no matching zero-percentage entry is found.

Discussion

Debugger022

Partially fixed.

- Unreleased income: documented in d6ae68a. `addOrUpdateDistributionConfigs` now says to call `releaseFunds` for the affected pools and assets first.
- Schema at 0%: acknowledged, documented in 59db3aa. No income is lost. It stays in `assetsReserves` and the first `releaseFunds` after the schema has targets pays it out. Not crediting the schema would be worse, since the next `updateAssetsState` for that asset, from any pool, would book it instead.
- Silent removal: fixed in 3ee0fc7. `removeDistributionConfig` now reverts with `DistributionConfigNotFound` or `NonZeroPercentage`.

Issue L-18: Small check gaps in `liquidateAccount` and the liquidity helpers [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-venus-protocol-sep-11th-2026/issues/23

Summary

A few checks around liquidation and liquidity are weaker than they look. None of them loses funds on its own.

1. `liquidateAccount`'s final "every borrow is zero" check uses the market list read before the orders, not after them as upstream does.
2. `liquidateAccount` bounds its loop with `_ensureMaxLoops(ordersCount / 2)`, which allows about twice the limit.
3. `getHypotheticalAccountLiquidity` ignores a hypothetical borrow in a market the account hasn't entered, so the view overstates liquidity.
4. `setMarketBorrowCaps` and `setMarketSupplyCaps` don't check that the market is listed.

Vulnerability Detail

Stale market list. `liquidateAccount` reads `getAssetsIn(borrower)` once at L784 and reuses it for the final check at L833-840:

```
VToken[] memory borrowerAssets = getAssetsIn(borrower);   // L784, before the orders
...
for (uint256 i; i < marketsCount; ++i) {                  // L835, same list after
↪  the orders
    (, uint256 borrowBalance, ) = _safeGetAccountSnapshot(borrowerAssets[i],
    ↪  borrower);
    if (borrowBalance != 0) revert NonzeroBorrowBalanceAfterLiquidation();
}
```

Upstream `Comptroller` calls `getAssetsIn` again for this loop. The list can only grow mid-liquidation with a token that has transfer hooks: the repayment triggers a callback, and a contract approved as the borrower's delegate calls `borrowBehalf` on a new market. That borrow still passes `preBorrowHook`'s collateral-factor check, so it's fully backed, but the "zero debt after `liquidateAccount`" guarantee no longer holds.

Loop bound. L813 checks `_ensureMaxLoops(ordersCount / 2)`, so a batch can have about twice `maxLoopsLimit` orders. Inherited from upstream.

Hypothetical view. In `getHypotheticalAccountLiquidity` (L1748-1756) the hypothetical borrow for `vTokenModify` is only added while looping over the account's entered markets. For a market the account hasn't entered, the borrow is left out and the result overstates

liquidity. The real borrow path enters the market first, so only the view is wrong, but off-chain tools use it to size borrows.

Cap setters. `setMarketBorrowCaps` and `setMarketSupplyCaps` (L1012-1055) accept any address, so a typo sets a cap on nothing without an error.

Impact

No loss. The final check in `liquidateAccount` is weaker than upstream's, the batch bound is looser than the limit suggests, the view can mislead integrators, and cap typos go unnoticed. Filed as Info.

Code Snippet

- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptroller.sol#L784-L840
- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptroller.sol#L813
- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptroller.sol#L1727-L1772
- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptroller.sol#L1012-L1055

Tool Used

Manual Review

Recommendation

- Call `getAssetsIn(borrower)` again before the final loop in `liquidateAccount`.
- Use `_ensureMaxLoops(ordersCount)`.
- In `getHypotheticalAccountLiquidity`, apply the effect for `vTokenModify` even when it isn't in the entered list.
- Check `markets[vToken].isListed` in both cap setters.

Discussion

Debugger022

Partially fixed. Taking the four parts in order:

- Stale market list: fixed in a14d365. `liquidateAccount` now calls `getAssetsIn(borrower)` again before the final check, as upstream does. 21cac0a adds a test: a borrow opened in a new market during the repayment makes `liquidateAccount` revert with `NonzeroBorrowBalanceAfterLiquidation`.
- Loop bound: intended, no change. Upstream changed this from `ordersCount` to `ordersCount / 2` on purpose in isolated-pools#237, so that the loop limit can't block a batch liquidation. Each order pairs one borrowed market with one collateral market, so clearing an account can take up to about twice as many orders as it has markets, and an account can't be in more markets than `maxLoopsLimit`. The orders come from the caller, so a longer batch only costs the caller gas.
- Hypothetical view: acknowledged, no code change. This is inherited unchanged from upstream: the base Comptroller and the Core pool also apply the hypothetical effects only to markets the account has entered. Only the view is affected. `preBorrowHook` adds the borrower to the market before the liquidity check, so a real borrow is always checked with the new debt included.
- Cap setters: acknowledged, no code change. This is inherited unchanged from upstream. A cap on an address that isn't listed has no effect: `preMintHook` and `preBorrowHook` check that the market is listed before they read its cap, and `PoolRegistry.addMarket` sets both caps when it lists a market. A mistyped address leaves the intended market's cap unchanged, and the `NewBorrowCap` / `NewSupplyCap` event names the address that was set. Adding the check to both setters would put the contract 166 bytes over the EIP-170 limit.

Issue L-19: Code and event hygiene in `SpokeComptroller` and the PSR [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-venus-protocol-sep-11th-2026/issues/24

Summary

Small things that make the contracts harder to read or monitor. None affects funds.

Vulnerability Detail

`SpokeComptroller`:

- `NewLiquidationIncentive` (L962) is reused for the pool fallback value, and has no market field, so decoders can't tell a pool-wide change from a market-specific one.
- `unlistMarket` emits nothing about the account memberships it leaves behind.
- Several setters emit an event even when the new value equals the old one.

`ProtocolShareReserve`:

- `_getUnderlying` (L592-598) is never called, so the `WBNB` and `vBNB` immutables are unused.
- `DistributionConfig.percentage` is documented as "without any scale", but it's in basis points (`MAX_PERCENT = 1e4`).
- `AssetReleased` doesn't include the comptroller, so releases can't be tied to a pool from events alone.
- `ReservesUpdated` is emitted for schemas that received nothing.
- `isMarketRegistered` doesn't mention that the core pool is exempt from the registry check.

Impact

Off-chain tools have a harder time rebuilding state from events, and the dead code and wrong comment can mislead reviewers. No funds are affected. Filed as Info.

Code Snippet

- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptroller.sol#L945-L963

- https://github.com/VenusProtocol/protocol-reserve/blob/a7b8f12de27657378d4173432852c70b000339e2/contracts/ProtocolReserve/ProtocolShareReserve.sol#L39-L44
- https://github.com/VenusProtocol/protocol-reserve/blob/a7b8f12de27657378d4173432852c70b000339e2/contracts/ProtocolReserve/ProtocolShareReserve.sol#L98-L104
- https://github.com/VenusProtocol/protocol-reserve/blob/a7b8f12de27657378d4173432852c70b000339e2/contracts/ProtocolReserve/ProtocolShareReserve.sol#L592-L598

Tool Used

Manual Review

Recommendation

Use a separate event (or add a market field) for pool-wide incentive changes, skip events on no-op writes, remove `_getUnderlying` and the unused immutables, fix the `percentage` comment, add the comptroller to `AssetReleased`, and document the core-pool exemption in `isMarketRegistered`.

Discussion

Debugger022

Partially fixed.

`SpokeComptroller`, acknowledged, no code change:

- `NewLiquidationIncentive` is kept from upstream so existing tooling decodes it the same way. Per-market changes emit `NewMarketLiquidationIncentive`.
- `unlistMarket` is unchanged from upstream. It can't emit per account, since the contract keeps no list of a market's members. `getAssetsIn` drops an unlisted market for every account.
- Setters emit on no-op writes, as upstream does. Each event carries the new value, so replaying them still gives the right state.

`ProtocolShareReserve`, in 59db3aa:

- `_getUnderlying` removed. `WBNB` / `vBNB` are kept so the constructor and getters don't change, and their NatSpec now says they're unused.
- `percentage` comment fixed to basis points.
- `isMarketRegistered` NatSpec now notes the core-pool exemption.

- `AssetReleased`: acknowledged. A new field changes the event topic and breaks indexers. Each release also emits `ReservesUpdated`, which carries the comptroller.
- `ReservesUpdated` for idle schemas: acknowledged, unchanged from upstream. The event shows the balance unchanged.

Issue L-20: `InvalidLiquidationThreshold` natspec describe the opposite condition [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-venus-protocol-sep-11th-2026/issues/25

Summary

The natspec of `InvalidLiquidationThreshold` in `SpokeComptrollerInterface.sol` say the error is thrown when the liquidation threshold exceeds the collateral factor. This is the other way around. In `setCollateralFactor` the error is thrown when the liquidation threshold is lower than the collateral factor, or when its bigger than 1e18. A threshold that exceed the collateral factor is actually the normal and valid config. So anyone who read the interface (integrators, offchain decoders, people preparing a VIP) will get the wrong idea about why there call reverted.

Code snippet

```
// isolated-pools/contracts/Spoke/SpokeComptrollerInterface.sol#L105-L106
    /// @notice Thrown when liquidation threshold exceeds the collateral factor
    error InvalidLiquidationThreshold();
```

```
// isolated-pools/contracts/Spoke/SpokeComptroller.sol#L897-L905
↪  (setCollateralFactor)
        // Ensure that liquidation threshold <= 1
        if (newLiquidationThresholdMantissa > MANTISSA_ONE) {
            revert InvalidLiquidationThreshold();
        }

        // Ensure that liquidation threshold >= CF
        if (newLiquidationThresholdMantissa < newCollateralFactorMantissa) {
            revert InvalidLiquidationThreshold();
        }
```

Recommendation

```
--- a/isolated-pools/contracts/Spoke/SpokeComptrollerInterface.sol
+++ b/isolated-pools/contracts/Spoke/SpokeComptrollerInterface.sol
@@ -104,3 +104,3 @@ interface SpokeComptrollerInterface is ComptrollerInterface {

-    /// @notice Thrown when liquidation threshold exceeds the collateral factor
+    /// @notice Thrown when liquidation threshold is lower than the collateral
↪  factor or greater than 1e18
```

```
    error InvalidLiquidationThreshold();
```

Discussion

Debugger022

Fixed in 6fdcb24.

Issue L-21: Storage comment say liquidation threshold must be between 0 and collateral factor, real range is collateral factor to 1 [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-venus-protocol-sep-11th-2026/issues/26

Summary

The comment on `Market.liquidationThresholdMantissa` in `SpokeComptrollerStorage.sol` say the value "Must be between 0 and collateral factor". The code enforce the reverse. `setCollateralFactor` revert with `InvalidLiquidationThreshold` when the threshold is under the collateral factor, and also when its above 1e18. So the valid range is collateral factor up to 1, not 0 up to collateral factor. Every value the comment call valid (except when it equal the collateral factor) is actually rejected by the contract.

Code snippet

```
// isolated-pools/contracts/Spoke/SpokeComptrollerStorage.sol#L58-L61
        //  Multiplier representing the collateralization after which the borrow is
        ↪  eligible
        //  for liquidation. For instance, 0.8 liquidate when the borrow is 80% of
        ↪  collateral
        //  value. Must be between 0 and collateral factor, stored as a mantissa.
        uint256 liquidationThresholdMantissa;
```

```
// isolated-pools/contracts/Spoke/SpokeComptroller.sol#L897-L905
↪  (setCollateralFactor)
        // Ensure that liquidation threshold <= 1
        if (newLiquidationThresholdMantissa > MANTISSA_ONE) {
            revert InvalidLiquidationThreshold();
        }

        // Ensure that liquidation threshold >= CF
        if (newLiquidationThresholdMantissa < newCollateralFactorMantissa) {
            revert InvalidLiquidationThreshold();
        }
```

Recommendation

```
--- a/isolated-pools/contracts/Spoke/SpokeComptrollerStorage.sol
+++ b/isolated-pools/contracts/Spoke/SpokeComptrollerStorage.sol
```

```
@@ -59,3 +59,3 @@ contract SpokeComptrollerStorage {
         //  for liquidation. For instance, 0.8 liquidate when the borrow is 80% of
         ↪  collateral
-        //  value. Must be between 0 and collateral factor, stored as a mantissa.
+        //  value. Must be between collateral factor and 1, stored as a mantissa.
         uint256 liquidationThresholdMantissa;
```

Discussion

Debugger022

Fixed in 6fdcb24.

Issue L-22: `exitMarket` comment say it "Return true" but the function return `NO_ERROR` which is 0 [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-venus-protocol-sep-11th-2026/issues/27

Summary

Inside `exitMarket` the comment say "Return true if the sender is not already in the market". The function return a `uint256` and in this branch it return `NO_ERROR`, which is defined as `0` in storage. There is no bool anywhere here. If anything, a reader who map the number to a boolean would read 0 as false, so the comment say the exact opposite of what caller get back.

Code snippet

```
// isolated-pools/contracts/Spoke/SpokeComptroller.sol#L262, L276-L281
    function exitMarket(address vTokenAddress) external override returns (uint256) {
        ...
        Market storage marketToExit = markets[address(vToken)];

        /* Return true if the sender is not already ‘in’ the market */
        if (!marketToExit.accountMembership[msg.sender]) {
            return NO_ERROR;
        }
```

```
// isolated-pools/contracts/Spoke/SpokeComptrollerStorage.sol#L120
    uint256 internal constant NO_ERROR = 0;
```

Recommendation

```
--- a/isolated-pools/contracts/Spoke/SpokeComptroller.sol
+++ b/isolated-pools/contracts/Spoke/SpokeComptroller.sol
@@ -277,3 +277,3 @@ contract SpokeComptroller is

-        /* Return true if the sender is not already ‘in’ the market */
+        /* Return NO_ERROR if the sender is not already ‘in’ the market */
         if (!marketToExit.accountMembership[msg.sender]) {
```

Discussion

Debugger022

Fixed in 6fdcb24.

Issue L-23: `preMintHook` natspec say "Not restricted" even though the supply allowlist restrict it [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-venus-protocol-sep-11th-2026/issues/28

Summary

This fork add a supply allowlist to `preMintHook`. When `isSupplyAllowlistEnabled[vToken]` is true, only a minter that is inside `isAllowedSupplier[vToken]` can mint, and everyone else revert with `SupplyNotAllowed`. But the natspec still say `@custom:access Not restricted`, which was copied as is from upstream Comptroller. The same fork did update the access tag of `preSeizeHook` for the liquidation allowlist, so this one look like it was simply forgotten.

Code snippet

```
// isolated-pools/contracts/Spoke/SpokeComptroller.sol#L321-L338
     * @custom:error SupplyNotAllowed error is thrown if the market's supply
     ↪  allowlist is enabled and the minter is
     *   not on it
     * @custom:error SupplyCapExceeded error is thrown if the total supply exceeds
     ↪  the cap after minting
     * @custom:access Not restricted
     */
    function preMintHook(address vToken, address minter, uint256 mintAmount)
    ↪  external override {
        ...
        if (isSupplyAllowlistEnabled[vToken] && !isAllowedSupplier[vToken][minter])
        ↪  {
            revert SupplyNotAllowed(vToken, minter);
        }
```

```
// isolated-pools/contracts/Spoke/SpokeComptroller.sol#L570-L571 (preSeizeHook,
↪  already updated)
     * @custom:access Not restricted while the liquidation allowlist is disabled,
     ↪  otherwise the liquidator has to be
     *   on it
```

Recommendation

```
--- a/isolated-pools/contracts/Spoke/SpokeComptroller.sol
+++ b/isolated-pools/contracts/Spoke/SpokeComptroller.sol
```

```
@@ -323,3 +323,4 @@ contract SpokeComptroller is
      * @custom:error SupplyCapExceeded error is thrown if the total supply exceeds
       ↪  the cap after minting
-     * @custom:access Not restricted
+     * @custom:access Not restricted while the market's supply allowlist is
 ↪  disabled, otherwise the minter has to
+     *   be on it
      */
```

Discussion

Debugger022

Fixed in 6fdcb24.

Issue L-24: New spoke markets start with open supply and no price protection, and nothing forces either to be set [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit2/2026-09-venus-protocol-sep-11th-2026/issues/30

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

When a market is listed in SpokeComptroller, its supply allowlist is off, and it cannot be turned on before listing. Also, if the asset was never configured in DeviationBoundedOracle, borrowing power is quietly calculated at the spot price. The listing flow checks neither, so a missed governance step leaves the market open or unprotected without any revert.

Vulnerability Details

Supply allowlist. `isSupplyAllowlistEnabled[vToken]` is false for every new market. `setSupplyAllowlistEnabled` and `setAllowedSupplier` both revert with `MarketNotListed` until the market is listed, so they can only run after `PoolRegistry.addMarket`. `addMarket` already sets a supply cap. From then until the allowlist is turned on, `preMintHook` lets any account supply up to the cap. If both steps are in one VIP there is no gap, but nothing makes sure of that.

Price protection. For borrow and redeem, SpokeComptroller prices collateral with `deviationBoundedOracle.getBoundedPricesView`. If the asset never got `setTokenConfig` in DeviationBoundedOracle, `isBoundedPricingEnabled` is false and the oracle returns the spot price for both collateral and debt. `setCollateralFactor`, which `addMarket` calls, only checks that the spot oracle returns a price. So a market can be listed with a collateral factor above zero and no bounded pricing.

Impact

A new spoke market accepts supply from any account until governance turns on its allowlist. Its collateral is valued at plain spot price if the asset was not added to DeviationBoundedOracle.

Code Snippet

SpokeComptrollerStorage.sol:

```
/// @notice Whether a market accepts supply only from the accounts on its supply
↪  allowlist. Keyed by market, and
/// disabled by default, so a newly listed market accepts supply from anyone.
mapping(address => bool) public isSupplyAllowlistEnabled;
```

SpokeComptroller.sol, `setSupplyAllowlistEnabled`:

```
function setSupplyAllowlistEnabled(address vToken, bool enabled) external {
    _checkAccessAllowed("setSupplyAllowlistEnabled(address,bool)");

    if (!markets[vToken].isListed) {
        revert MarketNotListed(vToken);
    }

    isSupplyAllowlistEnabled[vToken] = enabled;
    emit SupplyAllowlistEnabledUpdated(vToken, enabled);
}
```

SpokeComptroller.sol, `preMintHook`:

```
if (isSupplyAllowlistEnabled[vToken] && !isAllowedSupplier[vToken][minter]) {
    revert SupplyNotAllowed(vToken, minter);
}
```

SpokeComptroller.sol, `setCollateralFactor`:

```
// If collateral factor != 0, fail if price == 0
if (newCollateralFactorMantissa != 0 && oracle.getUnderlyingPrice(address(vToken))
↪  == 0) {
    revert PriceError(address(vToken));
}
```

DeviationBoundedOracle.sol, `_computeBoundedPrices`:

```
uint256 spot = _fetchSpotPrice(asset);
MarketProtectionState storage state = assetProtectionConfig[asset];
if (!state.isBoundedPricingEnabled) return (spot, spot);
```

Tools Used

Manual Review

Recommendation

Let the two allowlist setters work before the market is listed. They never call the vToken, so the `isListed` check is not needed there. The VIP can then allow the `vTokenReceiver` and turn the allowlist on before `addMarket`:

```
function setSupplyAllowlistEnabled(address vToken, bool enabled) external {
    _checkAccessAllowed("setSupplyAllowlistEnabled(address,bool)");
    ensureNonzeroAddress(vToken);

    isSupplyAllowlistEnabled[vToken] = enabled;
    emit SupplyAllowlistEnabledUpdated(vToken, enabled);
}
```

Do the same for `setAllowedSupplier`. In `setCollateralFactor`, when the new collateral factor is above zero, require bounded pricing to be enabled for the asset:

```
if (
    newCollateralFactorMantissa != 0 &&
    !deviationBoundedOracle.isBoundedPricingEnabled(vToken.underlying())
) {
    revert BoundedPricingNotEnabled(address(vToken));
}
```

Discussion

Debugger022

Acknowledged, no code change. Both parts are governance setup steps, and the listing proposal covers them.

- Supply allowlist: the listing proposal enables the liquidity market's allowlist right after `addMarket`, in the same proposal. A proposal executes all its actions in one transaction, so there is no block in which the market is listed with the allowlist off. The collateral markets are meant to be open to any supplier.
- Bounded pricing: an asset with no bounded-oracle config is priced at spot on both legs. This is documented on `_safeGetUnderlyingPrices` and covered by tests. Turning bounded pricing off for an asset is a valid setting, so we don't block it on-chain. A check in `setCollateralFactor` would also only hold at the moment it runs: `setAssetBoundedPricingEnabled` on the DeviationBoundedOracle can turn bounded pricing off later, and the comptroller wouldn't see it. Before listing, we confirm for each asset that bounded pricing is enabled or that spot pricing is intended.

Issue L-25: Supply allowlist can be skipped with mintBehalf because only the receiver is checked [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit2/2026-09-venus-protocol-sep-11th-2026/issues/31

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

The supply allowlist only checks the `minter`, which is the account that receives the vTokens. With `mintBehalf`, the caller pays for the mint but is never checked, and the receiver does not need to approve it. So an account that is not on the allowlist can still supply to the market by naming an allowlisted account as `minter`.

Vulnerability Details

`VToken.mintBehalf(minter, mintAmount)` takes the underlying from `msg.sender` and calls `_mintFresh(msg.sender, minter, mintAmount)`. `_mintFresh` sends only `minter` to `comptroller.preMintHook`, so the payer never reaches the comptroller.

`preMintHook` then checks `isAllowedSupplier[vToken][minter]` and nothing else. There is no approval step in `mintBehalf`, so any account can pick any allowlisted address as `minter` and supply to a market that should only take supply from allowlisted accounts.

Impact

Accounts that are not on the supply allowlist can still bring their funds into an allowlisted market and use up its supply cap, and the allowlisted account receives vTokens it never asked for.

Code Snippet

VToken.sol, `mintBehalf`:

```
function mintBehalf(address minter, uint256 mintAmount) external override
↪   nonReentrant returns (uint256) {
    ensureNonzeroAddress(minter);

    accrueInterest();

    _mintFresh(msg.sender, minter, mintAmount);
```

```
    return NO_ERROR;
}
```

VToken.sol, `_mintFresh`:

```
function _mintFresh(address payer, address minter, uint256 mintAmount) internal {
    /* Fail if mint not allowed */
    comptroller.preMintHook(address(this), minter, mintAmount);
```

SpokeComptroller.sol, `preMintHook`:

```
// `minter` is the account credited with the newly minted vTokens, not necessarily
↪  the account paying for
// them: `mintBehalf` lets a third party fund a mint attributed to someone else.
↪  Metering the recipient is
// what bounds the market's supply, so the payer is deliberately not checked.
if (isSupplyAllowlistEnabled[vToken] && !isAllowedSupplier[vToken][minter]) {
    revert SupplyNotAllowed(vToken, minter);
}
```

Tools Used

Manual Review

Recommendation

Check the payer too while the allowlist is on. Pass the payer into `preMintHook` and require both accounts to be allowlisted. Spoke markets run on their own VToken beacon, so this change does not have to reach other pools.

```
function _mintFresh(address payer, address minter, uint256 mintAmount) internal {
    /* Fail if mint not allowed */
    comptroller.preMintHook(address(this), payer, minter, mintAmount);
```

```
if (
    isSupplyAllowlistEnabled[vToken] &&
    (!isAllowedSupplier[vToken][minter] || !isAllowedSupplier[vToken][payer])
) {
    revert SupplyNotAllowed(vToken, minter);
}
```

Discussion

Debugger022

Acknowledged, no code change. On the liquidity market, the only account we put on the allowlist is the Hub's YieldGroup. A `mintBehalf` that names it is a donation: it costs the payer and only raises the value of existing Hub shares.

- Checking the payer needs `VToken` to pass it to `preMintHook`. The spoke beacon points to the same `VToken` contract that every isolated pool uses: `027` deploys `Spoke VTokenImpl` from `VToken`. This change would therefore need a spoke-only VToken with a different hook signature.
- A donation can use up supply-cap headroom, but the donor gives the full amount to Hub depositors to do it.
- The `preMintHook` comment already says the payer is deliberately not checked.

Issue L-26: The only allowlisted liquidator cannot be liquidated if it also borrows [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit2/2026-09-venus-protocol-sep-11th-2026/issues/32

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

When the liquidation allowlist is on, only allowlisted accounts can seize collateral, and VToken never lets an account liquidate itself. Borrowing is not allowlisted, so an allowlisted liquidator can also borrow. If it is the only liquidator on the list, nobody can liquidate or heal its position.

Vulnerability Details

`preSeizeHook` checks the account that receives the collateral with `_checkLiquidatorAllowlisted`, and `healAccount` checks `msg.sender` the same way. So while `isLiquidationAllowlistEnabled` is true, only accounts in `isAllowedLiquidator` can use `liquidateBorrow`, `liquidateAccount` or `healAccount`.

VToken also reverts when the liquidator and the borrower are the same account, in `_liquidateBorrowFresh` and in `_seize`. `healAccount` called by the borrower on itself reaches `_seize` with the same address and reverts too.

`preBorrowHook` has no allowlist check, so an allowlisted liquidator can open a borrow like anyone else. If it is the only account on the list, the only account allowed to liquidate it is itself, and that always reverts.

Impact

If the only allowlisted liquidator has a borrow, its position cannot be liquidated or healed when it becomes unhealthy, so it can grow into bad debt until governance adds another liquidator or turns the allowlist off.

Code Snippet

SpokeComptroller.sol, `preSeizeHook`:

```
// Every seizure reaches this hook with the account that receives the collateral,
↪  whichever entry point it
// came from, so this is the check that actually enforces the allowlist.
_checkLiquidatorAllowlisted(liquidator);
```

SpokeComptroller.sol, `_checkLiquidatorAllowlisted`:

```
function _checkLiquidatorAllowlisted(address liquidator) private view {
    if (isLiquidationAllowlistEnabled && !isAllowedLiquidator[liquidator]) {
        revert LiquidationNotAllowed(liquidator);
    }
}
```

VToken.sol, `_liquidateBorrowFresh`:

```
/* Fail if borrower = liquidator */
if (borrower == liquidator) {
    revert LiquidateLiquidatorIsBorrower();
}
```

VToken.sol, `_seize`:

```
/* Fail if borrower = liquidator */
if (borrower == liquidator) {
    revert LiquidateSeizeLiquidatorIsBorrower();
}
```

Tools Used

Manual Review

Recommendation

Do not let an allowlisted liquidator borrow in the pool while the liquidation allowlist is on. Add a check in `preBorrowHook`, and in `setAllowedLiquidator` refuse an account that already has a borrow:

```
if (isLiquidationAllowlistEnabled && isAllowedLiquidator[borrower]) {
    revert LiquidatorCannotBorrow(borrower);
}
```

Another option is to require at least two allowlisted liquidators before the allowlist can be turned on.

Discussion

Debugger022

Acknowledged, no code change.

- The liquidation allowlist is off by default, and the listing proposal doesn't enable it. Liquidation in this pool is meant to be permissionless.
- Reaching this needs governance to enable the allowlist with a single account on it, and that account, chosen by governance, to borrow and go into shortfall.
- Governance can recover by adding another liquidator or turning the allowlist off.
- Blocking allowlisted liquidators from borrowing doesn't fit under the contract size limit: even the smallest check in `preBorrowHook` puts the contract 31 bytes over.
- Mitigation: if the allowlist is ever enabled, we will put at least two accounts on it, and only liquidator contracts that don't borrow in the pool.

Issue L-27: # The adapter's `removeResource` emptiness gate reads a balance any third party can raise, so one vToken sent to the YieldGroup blocks deregistering a drained spoke market [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit2/2026-09-venus-protocol-sep-11th-2026/issues/33

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`AdapterSpokeV1.receiptBalance` returns the YieldGroup's raw vToken balance, and the adapter interface says `YieldGroup.removeResource` uses it as the emptiness gate: removal reverts while it is non-zero. That balance isn't controlled by the YieldGroup. Any account can credit vTokens to it without the YieldGroup doing anything:

- a plain `vToken.transfer(yieldGroup, 1)`: `preTransferHook` only checks that the sender may redeem, nothing about the recipient;
- `vToken.mintBehalf(yieldGroup, amount)`: open to anyone, and the YieldGroup is the allowed supplier.

So once the Hub drains a spoke market and governance queues `removeResource`, anyone can send one vToken to the YieldGroup first and the removal reverts. It costs about 2e10 wei of the underlying at a `2e28` rate and can be repeated every time.

Vulnerability Detail

The adapter defines the gate (`receiptBalance`, L349-351) and documents its use in `IResourceAdapter` (L117-126):

```
Used by `YieldGroup.removeResource` as a share-based emptiness gate. A value-based
check (`totalAssets`) rounds a small-but-non-zero receipt balance down to 0, which
would let removal orphan those receipt tokens; reading the raw balance closes that
↪  gap.
```

The gate was designed against the YieldGroup's own rounding leaving a residual. It doesn't consider tokens arriving from outside. Two ways they arrive, neither needing the YieldGroup's consent:

- `SpokeComptroller.preTransferHook` (L639-655) runs `_checkRedeemAllowed(vToken, src, ...)` and the reward hooks, and checks nothing about `dst`. Anyone holding a vToken of the market (the listing-seed receiver, any allowlisted supplier, a liquidator who received seized vTokens) can transfer one to the YieldGroup.

- `VToken.mintBehalf` (L294-300) credits `minter` and pulls from `msg.sender`. With `minter` set to the YieldGroup the supply allowlist passes, because the YieldGroup is on it.

Scenario:

1. Governance winds the spoke market down. Hub users withdraw until the YieldGroup's vToken balance is 0 (the flow `hubFlow.ts` tests as "lets a fully drained market be deregistered").
2. The removal is queued in the timelock, so its execution block is public.
3. Before it executes, anyone sends one vToken to the YieldGroup.
4. `removeResource` sees `receiptBalance != 0` and reverts. Governance has to route another withdrawal through the resource to empty it, and the attacker repeats step 3.

One limit: a single transaction that withdraws through the Hub and then calls `removeResource` clears anything that arrived before it, and nothing can land in between. The grief works against a removal that assumes the balance stays zero once drained, which is the tested flow.

Impact

Any account can keep a drained spoke market registered on the YieldGroup indefinitely, at dust cost per attempt. No funds are lost; the attacker donates the vTokens to Hub holders. Governance can still take the resource out of the inner queues and pause it, so the market stops being used, but it can't be deregistered through the documented flow.

Code Snippet

- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/test/HubSpoke/AdapterSpokeV1.sol#L348-L351
- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/test/HubSpoke/interfaces/IResourceAdapter.sol#L117-L126
- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/Spoke/SpokeComptroller.sol#L639-L655
- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/VToken.sol#L294-L300

Tool Used

Manual Review

Recommendation

Make removal robust to tokens arriving from outside rather than trying to stop every way they can arrive. For example, have the removal path redeem any remaining vTokens itself (through the adapter, with `redeem(balance)` for the full balance) before checking emptiness, or let `removeResource` treat a balance worth less than one unit of underlying as empty and sweep it to idle.

Discussion

Debugger022

Acknowledged. No code change planned for now.

- This is not spoke specific. The balance gate is in `YieldGroupBase.removeResource`, so it applies to every adapter.
- Mitigation: pause MINT and TRANSFER, drain the market with `reallocate`, then call `removeResource`. Both pauses are already required by `unlistMarket`, so this is part of the normal wind down.
- No funds are at risk, and any donated tokens go to Hub depositors.

Issue L-28: `AdapterSpokeV1.maxWithdraw` can certify an amount whose redeem burns one more vToken than the YieldGroup holds, so a withdrawal of exactly `maxWithdraw` reverts and the position can't be drained to zero [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-venus-protocol-sep-11th-2026/issues/34

Summary

`maxWithdraw` bounds its answer by the position's pro-rata value, `vBal * (cash + totalBorrows - totalReserves) / totalSupply`, then checks that the market's redeem arithmetic settles it within payable cash. It never checks the third constraint: that the vTokens the market burns for that request fit inside the holder's own balance. `redeemUnderlying` rounds its burn up and prices at the market's own exchange rate, which is itself rounded down, so the last unit of the pro-rata value can need one more vToken than the holder owns. The market then reverts with an arithmetic underflow in `accountTokens[redeemer] - redeemTokens`.

The adapter only ever calls `redeemUnderlying`, so the Hub has no path to that last unit. The YieldGroup's withdraw surfaces the failed leg as `ResourceLiquidityInsufficient` and the whole withdrawal reverts, and `removeResource`, which gates on `receiptBalance == 0`, can't be reached. The repo's own `tests/hardhat/Fork/HubSpoke/lowDecimals.ts` has a test for exactly this case, and it fails.

Vulnerability Detail

`maxWithdraw` (L301-333) computes:

```
uint256 ourValue = _recoverableValue(resource, vBal);          // vBal * (backing -
↪  reserves) / supply, floored
...
uint256 payableTokens = ((cash - reserves) * EXP_SCALE) / exchangeRate;
uint256 cashBound = (payableTokens * exchangeRate) / EXP_SCALE;
uint256 liquid = ourValue < cashBound ? ourValue : cashBound;
...
uint256 payout = _redeemPayout(exchangeRate, _bumpToSettleable(exchangeRate,
↪  liquid));
if (payout == 0 || payout > cash - reserves) return 0;
return liquid;
```

`_redeemPayout` mirrors `_redeemFresh`: `tokens = floor(request * 1e18 / rate)`, then `token`

`s + 1` if those tokens aren't worth exactly `request`. Nothing compares `tokens` with `vBal`.

The two roundings don't agree. `ourValue` is `vBal * B / S` rounded down once. The market's rate is `(B + badDebt) * 1e18 / S` rounded down, so it is slightly below the true per-token value, and `ourValue / rate` comes out slightly above `vBal`. After the round-up, the burn is `vBal + 1`.

Numbers from the repo's own low-decimal fixture (`addLowDecimalMarket`: 6-decimal TRX, 8-decimal vToken, listed at `3.7e16`, 1,000 TRX seed, then 100,000 TRX from the Hub):

	value
seed vTokens	27,027,027,027
Hub vTokens (`vBal`)	2,702,702,702,700
`totalSupply`	2,729,729,729,727
cash	101,000,000,000
`exchangeRateStored`	37,000,000,000,037,000
`ourValue` (pro-rata)	100,000,000,000
`cashBound`	100,999,999,999
`maxWithdraw`	100,000,000,000
vTokens burned by `redeemUnderlying(100,000,000,000)`	2,702,702,702,701

The burn is one vToken more than the Hub holds. `redeemUnderlying(maxWithdraw)` reverts in `_redeemFresh` at `accountTokens[redeemer] - redeemTokens`, and the Hub's withdraw of that amount fails.

The same numbers show a second thing. `vBal * exchangeRateStored / 1e18` is 99,999,999,999, so the market would pay 99,999,999,999 for every token the Hub owns, while `totalAssets` reports 100,000,000,000. The NatSpec on `totalAssets` (L229-231) says the mark "can never overstate what the position delivers". It does, by one unit, from the same rounding.

There is a second way the drain fails on these markets. Below `1e18`, a residual of vTokens worth less than one unit of underlying gives `ourValue == 0`, so `maxWithdraw` reports 0 and `redeemUnderlying` can never be asked for it. The adapter never calls `redeem(tokens)`, so such a residual is stuck for good. A full drain through the adapter therefore only works when the final burn lands exactly on `vBal`: one too many (this report's case) reverts, one too few leaves a residual that can't be reached.

The condition is `vBal * B` not dividing `S` evenly, which needs a second supplier. Every spoke market has one: `PoolRegistry.addMarket` mints the listing seed to `vTokenReceiver`.

It isn't limited to low-decimal markets. An 18-decimal market listed at `1e28`, with one seed and one deposit and no interest yet:

	value
seed	6,264,652,172,816,684,411,455,106
deposit	6,007,727,398,985,953,990,140,429,881
`vBal`	600,772,739,898,594,975
`exchangeRateStored`	10,000,000,000,000,007,057,794,305,147
`totalAssets` = `maxWithdraw`	6,007,727,398,985,953,990,130,422,344
full redeem of `vBal`	6,007,727,398,985,953,990,130,422,343
vTokens burned by `redeemUnderlying(maxWithdraw)`	600,772,739,898,594,976

Again one vToken more than held, and the mark one wei above a full redeem. Across 20,000 random seed and deposit amounts at `1e28`, about 2.5% of states end up like this; how often depends on how the amounts are spread. The fixture's round 10,000 / 100,000 amounts happen to miss it, which is why `hubFlow.ts` drains cleanly.

Impact

The Hub can't withdraw the full amount the adapter certifies, and the YieldGroup can't be emptied through the adapter, so `removeResource` is blocked for that market. It happens on 18-decimal markets as well, in a few percent of ordinary seed-plus-deposit states. On a market whose exchange rate is below `1e18` (an underlying with fewer decimals than the vToken, such as a 6-decimal stablecoin) the stuck residual is a separate, independent case on top of it. No funds are lost: the stuck amount is under one unit of underlying, and a smaller withdrawal works. It's a liveness gap in a function whose NatSpec says the floor "still lets the last vToken out, which is what lets a market be drained to zero and deregistered". The low-decimal case is also reproduced by the open test in `lowDecimals.ts` ("certifies a bound the holder's own vTokens can actually settle").

Code Snippet

- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/test/HubSpoke/AdapterSpokeV1.sol#L301-L333
- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/test/HubSpoke/AdapterSpokeV1.sol#L386-L399

- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/test/HubSpoke/AdapterSpokeV1.sol#L453-L458
- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/VToken.sol#L992-L1024
- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/tests/hardhat/Fork/HubSpoke/lowDecimals.ts#L92-L131

Tool Used

Manual Review

Recommendation

In `maxWithdraw`, after computing `liquid`, compute the burn the way `_redeemPayout` does and, if it exceeds `vBal`, lower `liquid` to the value of `vBal` whole tokens at the market rate (`vBal * exchangeRate / 1e18`, re-checked through `_redeemPayout`). Alternatively let `withdraw` fall back to `redeem(vBal)` when the request would burn the whole balance, so the last vToken can always leave. Apply the same `vBal * exchangeRate / 1e18` cap to `totalAssets` so the mark never exceeds what the market pays.

Discussion

Debugger022

Valid, fixed.

- **Over-burn:** already fixed. The adapter now redeems by vToken count (`redeem(_burnFor(...))`, in f81dd33), and the position is valued as `vBal * exchangeRateStored / 1e18` (in 6fe0224). A withdrawal of `maxWithdraw` can no longer burn more than the YieldGroup holds.
- **Dust on markets with a rate below 1e18:** fixed in bc3c5d5. When the vTokens left after the burn would be worth 0 underlying, `withdraw` now burns them together with the last withdrawal, so a full drain ends at a zero balance and `removeResource` succeeds.

Issue L-29: `AdapterSpokeV1.maxDeposit` keeps advertising room into a market that carries bad debt the live Shortfall can't auction, so every new deposit the Hub routes there raises the Hub's share of a loss it already marked [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-venus-protocol-sep-11th-2026/issues/35

Summary

The adapter reads `badDebt` to value the position (`totalAssets` excludes it, so the Hub marks a loss the moment `healAccount` writes it off), but `maxDeposit` ignores it. A market with bad debt still reports its full supply-cap headroom, so the Hub keeps routing new deposits into it. Each of those deposits mints vTokens at the market's exchange rate, which includes `badDebt` in its numerator, while the Hub marks them at the rate without it. The difference is a loss the Hub takes on the spot: its token share of the market grows, and so does its share of the debt nobody will repay. On a spoke pool that debt can't be auctioned: `deploy/028` wires the live `Shortfall`, which only knows the isolated-pools registry, so `startAuction` reverts for a spoke comptroller. Recovering it takes governance repointing `shortfall` with `setShortfallContract` and repaying it by hand through `badDebtRecovered`, so until that happens the hole is permanent.

Vulnerability Detail

Let `B` be `cash + totalBorrows - totalReserves`, `bd` the market's `badDebt`, `S` its `totalSupply`. The market mints at `R = (B + bd) / S` and the adapter marks at `r = B / S`.

A deposit `D` buys `D / R` vTokens, marked at `D * r / R < D`. The Hub's `totalAssets` rises by less than the `D` it took in, so its share price falls for everyone, the new depositor included. Nothing is wrong with the mark: the Hub's claim on the market's recoverable backing really did grow by less than `D`, because part of `D` now backs the share of the bad debt that moved from the market's other suppliers to the Hub.

With the numbers from `badDebt.ts` (Hub about 90% of the market, after a heal): the Hub's expected loss is `bd * T / S`, with `T` its vToken count. A deposit of `D` raises that by about `0.1 * D * bd / (B + bd)`. At 5% bad debt that's about 0.5% of every deposit the router sends in, moved from Hub depositors to the market's other suppliers, on every deposit, until governance notices and lowers the resource cap or pauses the resource. In general the leak per deposit is the other suppliers' share of supply times `D * bd / (B + bd)`. With the allowlist on, the only other supplier is the listing seed, so the fraction shrinks as the Hub becomes the market.

`maxDeposit` (L245-274) checks the allowlist, the MINT pause and the supply cap, and returns the cap headroom. It has `badDebt` one call away and doesn't look at it. A direct supplier to the market can see `badDebt` and choose; a Hub depositor can't, because the router chooses for them.

Impact

After any default in a spoke market, Hub depositors lose a fraction of every deposit routed into that market, with no automatic stop. It needs a default first and governance can stop it by hand, so it's Low. The in-scope point is that the adapter already treats `badDebt` as a loss on the valuation side and not on the routing side.

Code Snippet

- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/test/HubSpoke/AdapterSpokeV1.sol#L245-L274
- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/test/HubSpoke/AdapterSpokeV1.sol#L386-L399
- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/contracts/VToken.sol#L1664-L1681
- https://github.com/VenusProtocol/isolated-pools/blob/7b02a6082d213c01dc51614baba47f520eedf193/tests/hardhat/Fork/HubSpoke/badDebt.ts#L59-L74

Tool Used

Manual Review

Recommendation

Return 0 from `maxDeposit` while `IVTokenIsolated(resource).badDebt() != 0`, so the routing cascade skips the market until the debt is recovered (through a repointed `shortfall` and `badDebtRecovered`) or governance decides otherwise. If some tolerance is wanted, cap the room so that the Hub's token share of the market doesn't grow while `badDebt` is non-zero.

Discussion

Debugger022

Valid at the time of reporting, now fixed with a different approach.

- In 6fe0224, the adapter values spoke positions at the market's own exchange rate, which includes `badDebt`. The mint rate and the valuation rate are now the same, so a deposit into a market with bad debt no longer loses value when it lands.
- Spoke bad debt is repaid from the risk fund through a governance VIP, which points `shortfall` at the timelock and calls `badDebtRecovered`. The Shortfall auction limitation is the same point as #17.
- Because of this, `maxDeposit` does not need to return 0 while there is bad debt.

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.